

Prague University of Economics and Business
Faculty of Informatics and Statistics



**Development of a documentation
generating application for .NET
libraries**

BACHELOR THESIS

Study program: Applied Informatics

Field of study: Applied Informatics

Author: Denis Akopyan

Supervisor: Ing. Rudolf Pecinovský, CSc.

Prague, December 2022

Acknowledgements

Many thanks to Ing. Jarmila Pavlíčková, Ph.d. for supervising the majority of my thesis.

Abstrakt

Zaměřením dané bakalářské práce je výzkum a vývoj aplikace pro generování programátorské dokumentace .NET knihoven. Hlavním cílem bylo vytvoření nástroje pro .NET vývojáře, kteří chtějí extrahovat dokumentaci zdrojového kódu a exportovat ji do požadovaného výstupu, a zároveň zachovat aplikaci rozšiřitelnou, multiplatformní a uživatelsky přístupnou.

Teoretická část práce se bude zabývat plánováním vývoje a očekávaným výsledkem projektu.

Empirická část popíše vývoj samotný a zaměřuje se na tvorbu oddělené projektové struktury.

Očekávaným výsledkem bakalářské práce bude funkční, multiplatformní a open-source aplikace, která umožňuje generování dokumentace z .NET knihoven, a zároveň je rozšiřitelná prostřednictvím modulů.

Klíčová slova

vývoj, tvorba, aplikace, generování, dokumentace, dotnet, .NET, knihoven

JEL klasifikace

C88, O30

Abstract

This thesis aims to research and develop a documentation-generating application for .NET libraries. The main objective was to create a tool for .NET developers that want to extract source code documentation and export it into their desired format while keeping the application extensible, cross-platform, and easy to use.

The theoretical part of the thesis shall cover the development planning and the expected result of the project.

The empirical part of the thesis illustrates the exact process of development. Designing a decoupled project structure had the most attention dedicated to it.

The expected result of this thesis project is a working cross-platform and open-source application that can generate documentation from .NET libraries and is extensible via plugins.

Keywords

development, application, generating, documentation, dotnet, .NET, libraries

JEL classification

C88, O30

Contents

Introduction	12
Motivation	12
Goal	13
Proof of concept	13
Evaluations and project planning	14
Processing libraries	14
GUI application	14
Markdown for Git plugin	15
1 Short overview of .NET	16
1.1 .NET Framework	16
1.2 .NET Core and .NET	16
1.3 .NET Standard	16
2 .NET documentation	18
2.1 Concept	18
2.2 Documentation tags	19
2.2.1 General tags	19
2.2.2 Members tags	20
2.2.3 Formatting tags	21
2.2.4 Documentation reuse tags	24
2.2.5 Reference tags	25
2.2.6 Generic types tags	26
3 Documentation tools	27
3.1 What is available?	27
3.1.1 DocFX	27
3.1.2 Doxygen	27
3.1.3 Source Browser	28
3.1.4 Vsxmd	28
3.1.5 Vsd2md	28
3.2 What do users want?	29
3.2.1 Questionnaire results	30
3.2.2 Questionnaire evaluation	34
4 Proof of concept	35
4.1 Milestone goals analysis	35
4.1.1 Documentation extraction methods	35
4.1.2 Open-source project hosting platforms	36
4.1.3 Parsing data into markdown	37

4.1.4	Development goals summary	37
4.2	Development results	37
4.2.1	Reflection for data extraction	37
4.2.2	Design pattern importance	38
4.2.3	Git platforms for hosting open-source projects	38
4.2.4	Git platforms for hosting Markdown documentation	38
4.2.5	Markdown generator	39
5	Evaluation and project planning	40
5.1	Evaluation	40
5.1.1	Project hosting platform	40
5.1.2	Technology stack	40
5.2	Development planning	42
5.2.1	Project architecture	42
5.2.2	Interfaces	43
5.2.3	GUI application	45
5.2.4	Plugin structure	45
5.2.5	Development stages	45
5.3	Project name	46
6	Processing libraries	47
6.1	Reflection-based member provider	47
6.1.1	Separation of types	47
6.1.2	Dynamic types and value types	48
6.1.3	Missing features	48
6.2	XML documentation provider	49
6.2.1	Tags documentation	49
6.2.2	Dependency on the member provider	50
6.3	Linker	50
6.3.1	Git-specific logic	50
6.4	Markdown elements	50
6.5	MermaidJS diagram generator	50
6.6	Composer	51
6.6.1	Composed structure	51
6.7	Printer	52
7	GUI application	53
7.1	AvaloniaUI	53
7.2	Decoupling via MVVM	53
7.3	Application structure	54
7.3.1	Home page	54
7.3.2	Settings page	56
7.3.3	About page	56
7.3.4	Configuration wizard	57
7.3.5	Summary page	58

8	Markdown for Git plugin	59
8.1	Plugin structure	59
8.2	Plugin configuration steps	60
8.2.1	Member provider	61
8.2.2	Documentation provider	62
8.2.3	Linker	63
8.2.4	Global configuration	64
9	Performance	65
9.1	Configurations	65
9.1.1	ModularDoc	65
9.1.2	Doxygen	65
9.1.3	DocFX	66
9.2	Results	66
10	Tool usage and extension guides	67
10.1	User guide	67
10.1.1	Prerequisites	67
10.1.2	ModularDoc application	68
10.1.3	Markdown for Git plugin	74
10.1.4	Plugin installation and management	77
10.2	Developer guide	78
10.2.1	Prerequisites	78
10.2.2	Accessing the source code	78
10.2.3	Developing a new component	79
10.2.4	Developing a new plugin	80
	Summary	81
	Goals and their fulfillment	81
	Open-source	81
	Personal achievements	81
	Future plans	82
	Bibliography	83
	Appendices	87
	ModularDoc compiled	87
	ModularDoc source code	87

List of Figures

1	Overplanning visualized	14
3.1	What output formats do you wish to have for your documentation?	30
3.2	Would you integrate such a tool in your CI/CD process?	30
3.3	How crucial is it for the tool to support these platforms?	31
3.4	Would it be beneficial to have a GUI instead of only a console?	31
3.5	Would you appreciate the tool being extensible via plugins?	32
3.6	Are you using tools for documentation generation?	32
3.7	What documentation generating tools do you use?	33
3.8	Would you be interested in contributing to this project?	34
5.1	Application architecture	43
7.1	Mock of the home page	55
7.2	Mock of the home page with a plugin selected	55
7.3	Mock of settings page	56
7.4	Mock of configuration wizard	57
7.5	Mock of summary page	58
8.1	Plugin components structure	59
8.2	Mock of the assembly configuration step	61
8.3	Mock of the documentation configuration step	62
8.4	Mock of the linker configuration step	63
8.5	Mock of the global configuration step	64
10.1	ModularDoc simplified file and folder structure	67
10.2	ModularDoc home page	68
10.3	ModularDoc home page with a plugin selected	69
10.4	ModularDoc configuration wizard	70
10.5	ModularDoc summary	71
10.6	ModularDoc summary log grouping	72
10.7	ModularDoc settings page	73
10.8	Plugin assembly configuration step	74
10.9	Plugin documentation configuration step	75
10.10	Plugin linker configuration step	76
10.11	Plugin global configuration step	77
10.12	ModularDoc simplified source code structure	79

Listings

2.1	C# documentation example	18
2.2	Visual Basic documentation example	18
2.3	F# documentation example	18
2.4	Summary tag	19
2.5	Remark tag	19
2.6	Returns tag	20
2.7	Param tag	20
2.8	Paramref tag	20
2.9	Exception tag	21
2.10	Value tag	21
2.11	Para tag	21
2.12	List tag	22
2.13	C tag	23
2.14	Code tag	24
2.15	Example tag	24
2.16	Inheritdoc tag	24
2.17	See tag	25
2.18	Seealso tag	26
2.19	Typeparam tag	26
2.20	Typeparamref tag	26
6.1	C# tuple and value tuple examples	48
10.1	Script for cloning the ModularDoc project GitHub repository	78

List of abbreviations

API Application Programming Interface	SOLID Single Responsibility Principal, Open-Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, Dependency Inversion Principle (18)
ASCII American Standard Code for Information Interchange (36)	
CI/CD Continuous integration and continuous deployment	
CLI Command Line Interface	TFS Team Foundation Server
DI Dependency Injection (12)	UI User Interface
GUI Graphical User Interface	URL Uniform Resource Locator
HTML Hypertext Markup Language	UX User eXperience
MVVM Model View ViewModel, structural design pattern (23)	WPF Windows Presentation Framework (39)
OS Operating System	XAML Extensible Application Markup Language
PDF Portable Document Format	XML eXtended Markup Language
RTF Rich Text Format	

Glossary

ASCII art text-based images consisting of ASCII characters (36)

DevOps a combination of development and operations that serve organizations for delivering their applications and services fast and with high quality (3)

Git A free and open-source distributed version control system designed to handle everything from small to substantial projects with speed and efficiency (15)

Markdown A popular lightweight markup language that is used to add formatting elements to plaintext text documents (10)

.NET A free, cross-platform, open-source developer platform for building many different types of applications (32)

ASP.NET Core A cross-platform, high-performance, open-source framework for building modern, cloud-enabled, Internet-connected apps (4)

NuGet Package manager for .NET projects (31)

Windows Forms A.k.a. WinForms, a GUI framework from Microsoft bundled with .NET (43)

Introduction

Source code documentation is an essential part of any quality project (35). Unfortunately, writing and keeping such documentation up to date is often overlooked or inconvenient for various reasons, such as strict deadlines. That might lead to the degradation of project quality, as developers who leave said projects usually take their know-how with them without properly passing it down to their replacements.

In cases where source code documentation has a higher priority than usual, extracting said documentation from the source code into a searchable, public format such as HTML is good practice (38). Documentation-generating tools accomplish that. The primary benefit of this practice is the resulting comprehensive overview of the source code, accessible to interested parties (38).

For Microsoft .NET projects, there is a selection of documentation-generating tools available, which includes: DocFX, Doxygen, SourceBrowser, and others (45). These solutions primarily generate output in HTML, which is sufficient for most projects. However, these tools often lack support for output formats like PDF, Markdown, or custom. Moreover, the customization of these tools is minimal, preventing projects from being able to fit such tools to their exact needs. Furthermore, their UI/UX leaves much to be desired, introducing an unnecessary learning curve and lowering usability.

With that in mind, there is visible room for improvement. A desirable documentation-generating tool would be easy to use, modern, extensible, rich in output format support, and performant. Satisfying the desires of a modern extensible application requires the thorough use of programming design patterns (20) such as SOLID (18), DI (12), and MVVM (23).

Motivation

The motivation for creating a custom tool is primarily a personal need to provide easy access to code documentation. Since most open-source projects are hosted either on GitHub.com or GitLab.com (2), it makes sense to utilize said platforms built-in wiki pages for hosting source code documentation for consistency. However, a minority of developers use Bitbucket to host their public open-source projects (22), as this platform targets the enterprise market (21). Additionally, corporate clients who purchase Bitbucket usually consider using Confluence alongside it, which serves as a documentation hosting platform. That is because both products are from the same vendor, Atlassian (21). Therefore, directly supporting Bitbuckets wiki is not a priority; however, adding future support for Confluence is possible, given the tool's extensibility.

When attempting to find existing solutions for generating documentation, none had the desired extensibility and were mainly limited to creating static HTML pages. Since the central Git

hosting platform's wiki pages predominantly utilize Markdown for displaying formatted text¹, static HTML pages are out of the question.

Thus, the idea of developing a custom tool to create Markdown documentation from .NET libraries for Git platforms came to fruition. Nevertheless, focusing only on one output format would waste time and effort, as only some would need such a tool. Thus, the result of the development should be a generic tool that allows anyone to modify it to output any desirable format.

Conducting a questionnaire to identify developer needs for documentation-generating tools in the .NET world supported this motivation. The questionnaire result (see section 3.2) confirmed a generally low interest in writing documentation; however, it highlighted the desires of those few developers who care about maintaining source code documentation.

Goal

This thesis aims to create a custom documentation-generating tool for .NET projects using appropriate design patterns such as SOLID, DI, and MVVM, and to satisfy user needs for extensibility, ease of use, modern design, and support for many output formats. Furthermore, reaching the goal must not introduce worse performance than existing tools.

Based on an analysis of currently available documentation tools (see section 3.1), developer needs (see section 3.2), personal experience, and general project development guidelines, the following milestones are defined:

1. Proof of concept
2. Evaluation and project planning
3. Processing libraries
4. GUI application
5. Markdown for Git plugin

Achieving each one is a step closer to the end goal of a functioning custom documentation-generating tool.

Proof of concept

Creating a proof of concept program will help understand whether the project is realistic, what challenges will occur and how to overcome them. The main drawback of this is that it will take additional time; however, the gained perspectives are quintessential for the correct project planning.

¹Apart from Markdown, said Git platforms support more formats; however, the latter has the richest formatting capabilities

The focus of this milestone is to find out how to extract the necessary data for documentation generation and attempt to generate said documentation. That is needed because no existing reference project could provide the required guidelines in a reasonable amount of time. All other requirements based on the takeaways from the survey (see 3.2.2) can be omitted from this stage, as reference projects and personal experience is available.

Evaluations and project planning

Concise planning of the project, based on the gained experience from the proof of concept, will, as a result, yield a higher quality product. Extensibility will be the key feature of the project. Therefore, careful planning of the code structure is required to avoid unnecessary complications. Meanwhile, the planning phase should not take unreasonable effort, as the ratio of time spent to results will diminish over time (as seen in figure 1) (37).

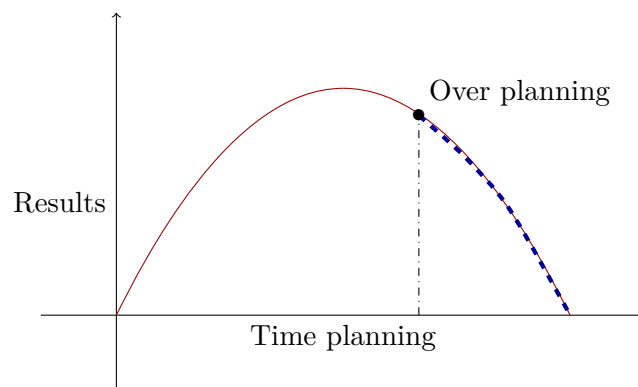


Figure 1: Overplanning visualized

This milestone aims to gain a clear picture of what techniques, design patterns (20), and technologies should be used or avoided to satisfy all project requirements with minimum compromise.

Processing libraries

The focus of this milestone will be creating processing libraries that will serve as the application's core. Said libraries will take some input and provide the output necessary for generating the desired documentation. The types of libraries and their implementation depend on the previous milestones. The result of this milestone will be a set of working libraries.

GUI application

The GUI application will serve as the façade for users utilizing the tool. Said application must be cross-platform, have a modern design, and provide as seamless a user experience as

possible, considering the extensible nature of the tool.

Thus, this milestone aims to create such a GUI façade that will have a simple, yet functional UX/UI. Additional goals might be added based on the outcomes of previous milestones.

Markdown for Git plugin

Creating the first plugin for the tool will be the final milestone for the time being as its creation reaches the goal of this thesis. The plugin will be composed of the processing libraries created in the fourth milestone.

The goal is to create a plugin that can be distributed separately from the main application (24). That would allow users to choose what plugins they wish to use and omit to install ones they do not.

1. Short overview of .NET

.NET is a free developer platform for building various types of applications. With it, developers can use multiple languages, editors, and libraries to build for the web, mobile, desktop, and more (32). Officially supported languages include C#, F#, and Visual Basic, C# being the dominant option (40).

The developer platform has evolved over the years, split into different flavors, and renamed, inevitably confusing developers. The following are the official .NET versions:

- .NET Framework.
- .NET Standard.
- .NET Core.
- .NET.

1.1 .NET Framework

The first platform was .NET Framework. It is a proprietary, free-to-use developer platform, supporting the Windows OS (9). Microsoft no longer focuses on said platform and will only provide minor fixes if necessary (9). The last major version, 4.8, was released on the 18th of April 2019. The latest minor version, 4.8.1, was released on the 9th of August 2022 (28).

1.2 .NET Core and .NET

.NET Core is the new open-source, cross-platform development framework from Microsoft (9). Its initial versions focused on web development with ASP.NET Core. However, starting from version 3.0, Microsoft has ported to it their desktop-based GUI frameworks (WPF and Windows Forms) from .NET Framework (13).

Versions following .NET Core 3.1 drop the *Core* keyword and start the version number from 5.0. That is to indicate that .NET 5.0 and higher is the next iteration after .NET Framework 4.8 (25).

1.3 .NET Standard

.NET Standard was used for sharing code between .NET Core and .NET Framework, as said frameworks are not cross-compatible (9). .NET Standard had multiple releases, each exposing a more significant subset of shared APIs between .NET Core and .NET Framework (9).

Microsoft has considered .NET Standard obsolete since the release of .NET 5.0. Microsoft incentivizes developers to migrate all their projects to the new framework; thus, eliminating the need for .NET Standard (26).

2. .NET documentation

A complete understanding of the .NET developer documentation system is essential for proper planning and development. This chapter describes the concept of .NET documentation in general and analyzes each type of documentation tag.

2.1 Concept

Source files of a language based on the .NET platform can have structured comments that produce documentation for types and members defined in said source files (45). The respective language compiler (C# and VB are compiled by Roslyn, while F# has its own specific F# compiler) produces an XML file that contains structured data representing comments bound to types and members.

Documentation comments start with `///` (or with `'''` for Visual Basic) followed by XML tags.

Listing 2.1: C# documentation example

```
1    /// <summary>
2    /// This is a brief description of the class
3    /// </summary>
4    public class MyClass
5    {
6        /// <summary>
7        /// This method executes some stuff
8        /// </summary>
9        public void Foo()
10       {
11           Bar();
12       }
13   }
```

Listing 2.2: Visual Basic documentation example

```
1    ''' <summary>
2    ''' This is a brief description of a procedure
3    ''' </summary>
4    Public Sub Foo()
5        Call Bar()
6    End Sub
```

Listing 2.3: F# documentation example

```
1    /// <summary>
2    /// This is a brief description of the class
3    /// </summary>
4    type MyClass =
```

```
5      /// <summary>
6      /// This function executes some stuff
7      /// </summary>
8      member _.Foo() = Bar
```

All following examples will be in C#, as it is the dominant language of the platform (40).

2.2 Documentation tags

The following is a list of supported XML tags for documenting source code. Tags described as root tags are top-level; therefore, they cannot exist within other documentation tags. Paired tags can contain a subset of other tags and text values. Meanwhile, non-paired tags can only provide additional information via their attributes. Finally, style tags change the way the documentation text is displayed.

The following examples are based on Microsoft's documentation (45) and Richard Carr's blogpost (8).

2.2.1 General tags

General tags can document both types and their members.

Summary

- Contains a brief description of the documented type or member
- Root tag
- Paired tag

Listing 2.4: Summary tag

```
1      /// <summary>
2      /// This is a brief description of the class
3      /// <c>An example of a code formatting tag within the summary tag</c>
4      /// </summary>
```

Remarks

- Provides additional information (remarks) about the documented type or member
- Root tag
- Paired tag

Listing 2.5: Remark tag

```
1    /// <remarks>
2    /// This is a remark
3    /// <c>An example of a code formatting tag within the remark tag</c>
4    /// </remarks>
```

2.2.2 Members tags

Member tags can document type members only.

Returns

- Documents the returned type by the member
- Root tag
- Paired tag

Listing 2.6: Returns tag

```
1    /// <returns>
2    /// The number of items
3    /// </returns>
4    int GetItemCount();
```

Param

- Documents a given method parameter
- Root tag
- Paired tag

Listing 2.7: Param tag

```
1    /// <param name="bar">Instance of bar to process</param>
2    void Foo(Bar bar);
```

Paramref

- References a methods parameter
- Can reference a parameter only in the context of the same method
- Useful when the given parameter name is changed

Listing 2.8: Paramref tag

```
1    /// <summary>
2    /// Loads data from <paramref name="bar"/>
3    /// </summary>
4    /// <param name="bar">Instance of bar to process</param>
```

```
5 void Foo(Bar bar);
```

Exception

- Warns that a given method throws an exception
- Root tag
- Paired tag, can be unpaired

Listing 2.9: Exception tag

```
1  /// <exception cref="FileNotFoundException"/>
2  /// <exception cref="ArgumentNullException">Throw when <paramref="bar"
    /// is found to be <c>null</c></exception>
3  void Foo(Bar bar);
```

Value

- Describes the value represented by the property
- Root tag
- Paired tag

Listing 2.10: Value tag

```
1  /// <value>
2  /// Numeric representation of users age
3  /// </value>
4  int Age { get; }
```

2.2.3 Formatting tags

Formatting tags are used style and format the documentation text for improved readability.

Para

- Indicates that a new line (new paragraph) should be inserted into the documentation text
- Style tag
- Paired tag, can be unpaired

Listing 2.11: Para tag

```
1  /// <summary>
2  /// Some text<para/>this text shall be on a new line
3  /// </summary>
4  void Foo();
```

List

Description of the list tag is based on Richard Carr's post (8).

- Complex style tag for creating:
 - Bullet lists
 - Numbered lists
 - Definition lists (either a bullet or numbered list of terms and their definitions, where the term is highlighted)
 - Tables
- Paired tag

Uses `<item/>`, `<term/>`, `<description/>`, and `<listheader/>` tags within itself to describe the list or table.

Listing 2.12: List tag

```
1    /// <summary>
2    /// This method can do one of the following:
3    /// <list type="bullet">
4    /// <item>
5    /// <description>Do nothing</description>
6    /// </item>
7    /// <item>
8    /// <description>Throw an exception</description>
9    /// </item>
10   /// <item>
11   /// <description>Do what was intended</description>
12   /// </item>
13   /// </list>
14   /// </summary>
15   void BulletListDoc();
16
17   /// <summary>
18   /// This method performs the following steps:
19   /// <list type="number">
20   /// <item>
21   /// <description>Initializes class</description>
22   /// </item>
23   /// <item>
24   /// <description>Extracts data</description>
25   /// </item>
26   /// <item>
27   /// <description>Starts the process</description>
28   /// </item>
29   /// </list>
30   /// </summary>
31   void NumberedListDoc();
32
33   /// <summary>
34   /// This method can be executed in the following scenarios:
```

```

35     /// <list type="bullet">
36     /// <item>
37     /// <term>Initialization</term>
38     /// <description>Program start</description>
39     /// </item>
40     /// <item>
41     /// <term>Termination</term>
42     /// <description>Program end</description>
43     /// </item>
44     /// </list>
45     /// </summary>
46     void DefinitionListDoc();
47
48     /// <summary>
49     /// Transforms location based on active direction
50     /// <list type="table">
51     /// <listheader>
52     /// <term>Direction</term>
53     /// <term>Description</term>
54     /// </listheader>
55     /// <item>
56     /// <term>Forward</term>
57     /// <term>Move forwards in a straight line.</term>
58     /// </item>
59     /// <item>
60     /// <term>Backward</term>
61     /// <term>Move backwards in a straight line.</term>
62     /// </item>
63     /// </list>
64     void TableDoc();

```

C

- Indicates that a given part of the text should be styled as inline code
- Style tag
- Paired tag

Listing 2.13: C tag

```

1     /// <summary>
2     /// This is code <c>var i = 5;</c>
3     /// </summary>
4     void Foo();

```

Code

- Indicates that a given part of the text should be styled as a code block
- The code block is automatically started on a new line

- Style tag
- Paired tag

Listing 2.14: Code tag

```

1      /// <remarks>
2      /// This method should be called before initialization:
3      /// <code>
4      /// Foo();
5      /// Initialize();
6      /// </code>
7      /// </remarks>
8      void Foo();

```

Example

- Specifies an example usage of a given type or member
- Paired tag
- Style tag

Listing 2.15: Example tag

```

1
2      /// <example>
3      /// This is some example
4      /// <code>
5      /// Foo();
6      /// </code>
7      /// </example>
8      void Foo();

```

2.2.4 Documentation reuse tags

Reuse tags allow copying documentation from a specific source.

Inheritdoc

- Copies documentation from the base type
- Root tag
- Unpaired tag

Listing 2.16: Inheritdoc tag

```

1      public interface IFirst
2      {
3          /// <summary>
4          /// This will be copied over via inheritdoc

```

```

5      /// </summary>
6      void Foo();
7
8      /// <summary>
9      /// This can also be copied over via inheritdoc
10     /// </summary>
11     void Bar();
12 }
13
14 public interface ISecond
15 {
16     /// <summary>
17     /// This will be copied because it was explicitly chosen
18     /// </summary>
19     void Bar();
20 }
21
22 public class MyClass : IFirst, ISecond
23 {
24     /// <inheritdoc/>
25     public void Foo();
26
27     /// <inheritdoc cref="ISecond"/>
28     public void Bar();
29 }

```

2.2.5 Reference tags

Reference tags allow creating references to other documented types and members, or create hyperlinks.

See

- Creates a link to a member or website
- Used to link from within the text
- Paired tag, can be unpaired

Listing 2.17: See tag

```

1      /// <summary>
2      /// This is some method, what follows are references:
3      /// <para/>
4      /// <see cref="MyNamespace.BarType"/>
5      /// <para/>
6      /// <see href="https://example.com">Link example</see>
7      /// </summary>
8      BarType Foo();

```

Seealso

- Creates a link to a member or website
- Used to place given link in a see also section
- Root tag
- Paired tag, can be unpaired

Listing 2.18: Seealso tag

```
1    /// <summary>
2    /// This is some method
3    /// </summary>
4    /// <seealso cref="MyNamespace.BarType"/>
5    /// <seealso href="https://example.com">Link example</seealso>
6    BarType Foo();
```

2.2.6 Generic types tags

Generic type tags allow documenting and referencing generic type parameters.

Typeparam

- Documents a given type of method generic parameter
- Root tag
- Paired tag

Listing 2.19: Typeparam tag

```
1    /// <typeparam name="T">Type of input</param>
2    void Foo<T>(T bar);
```

Typeparamref

- References a type or method generic parameter
- Can reference a generic parameter only in the context of the same type or method
- Useful when the given generic parameter name is changed

Listing 2.20: Typeparamref tag

```
1    /// <summary>
2    /// Loads data from <paramref name="bar"/> of a given type <
3    ///     typeparamref name="T"/>
4    /// </summary>
5    /// <typeparam name="T">Type of input</param>
6    void Foo<T>(T bar);
```

3. Documentation tools

Analysis of existing tools and user needs must precede project planning and development. This chapter attempts to answer these two questions:

1. What is available?
2. What do users want?

3.1 What is available?

The current documentation tool market provides the following solutions:

- DocFX.
- Doxygen.
- Source Browser.
- Vsxmd.
- vsdoc-2-md.

What follows are evaluations of each tool that provides prospects on features and improvements the custom tool utilize.

3.1.1 DocFX

DocFX is an open-source documentation generation tool developed by Microsoft that supports .NET languages, Java, JavaScript, TypeScript, Python, and others. Additionally, it can use raw Markdown files as input (42).

However, *DocFX* only outputs static HTML pages. Therefore, the only available customizability is via templates for said static pages (42).

The tool is capable and is the go-to solution for .NET projects, but it does not solve the task of outputting Markdown files.

Configuring and running *DocFX* is unintuitive, as it provides no GUI. Instead, it uses a configuration file. Moreover, the documentation for the tool leaves much to be desired, as it does not describe all possible settings.

3.1.2 Doxygen

Doxygen is the industry-standard documentation-generating tool originally made for C/C++ source code documentation (19). Support for popular programming languages, such as C#,

Java, Python, and many more, was added later.

Based on its description (19), the tool provides an extensive set of supported output formats:

- Static HTML.
- L^AT_EX.
- Man pages.
- RTF.
- XML.

It is possible to extend the tool, as it is open-source and available on GitHub. Unfortunately for .NET developers, *Doxygen* is written in C++, and its documentation does not provide clear guidelines for extending its support for output formats. Naturally, a curious programmer can deduce how to do that from the source code; however, this involves investing considerable time and effort.

Thankfully, configuring and executing *Doxygen* is made simple with its GUI.

3.1.3 Source Browser

Source Browser is a tool that generates a website for browsing source code and its documentation (33). The tool is used by Microsoft, for example, to allow developers to browse the source code of .NET (33).

The generated output is HTML and has to be hosted on an ASP.NET Core website to support searching (33).

3.1.4 Vsxmd

Vsxmd generates a single Markdown file for all types in a given assembly (34). Moreover, the tool has no UI and works as a NuGet package that the project designated for documentation generation references (34). Thus, configuring this tool is done via .NET project settings (34).

It is impossible to navigate the documentation as the tool does not generate links.

3.1.5 Vsdoc-2-md

Vsdoc-2-md is an entirely unusable tool for large projects, as it is purely web-based and generates documentation from the provided XML documentation source file (41). It is designed with the intent of quickly generating documentation of an API; thus, the tool is limited to processing only one XML file at a time (41).

Just like *Vsxml*, this tool does not generate links; thus, navigating the documentation is impossible.

3.2 What do users want?

Surveying potential tool users is necessary to ensure that the developed product is utilized by more than its developer. Thus, creating a questionnaire was the first step to success. Its purpose was to figure out the following key points:

- Whether users already use documentation generation tools, and if so, which ones?
- What do users feel is missing from the tools they use?
- What do users believe should be carried over to the new tool from the ones they use?
- Whether users would integrate such tools into their CI/CD pipelines?
- What output formats do users want?
- What operating systems do users want the tool to work on?
- Would users find a GUI beneficial for such a tool?
- Would users appreciate the tool being extensible via plugins?
- Would users be interested in contributing to the project?

3.2.1 Questionnaire results

What follows are the responses of the 22 participants of the questionnaire.

What output formats do you wish to have for your documentation?

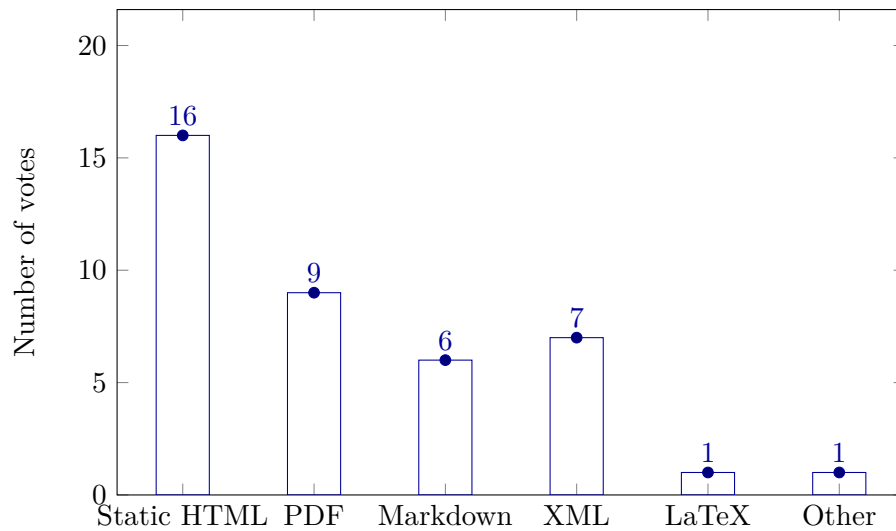


Figure 3.1: What output formats do you wish to have for your documentation?

Other responses:

- AsciiDoc.

Would you integrate such a tool in your CI/CD process?

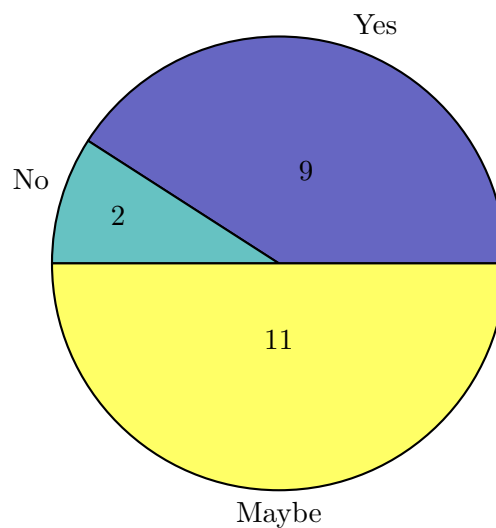


Figure 3.2: Would you integrate such a tool in your CI/CD process?

How crucial is it for the tool to support these platforms?

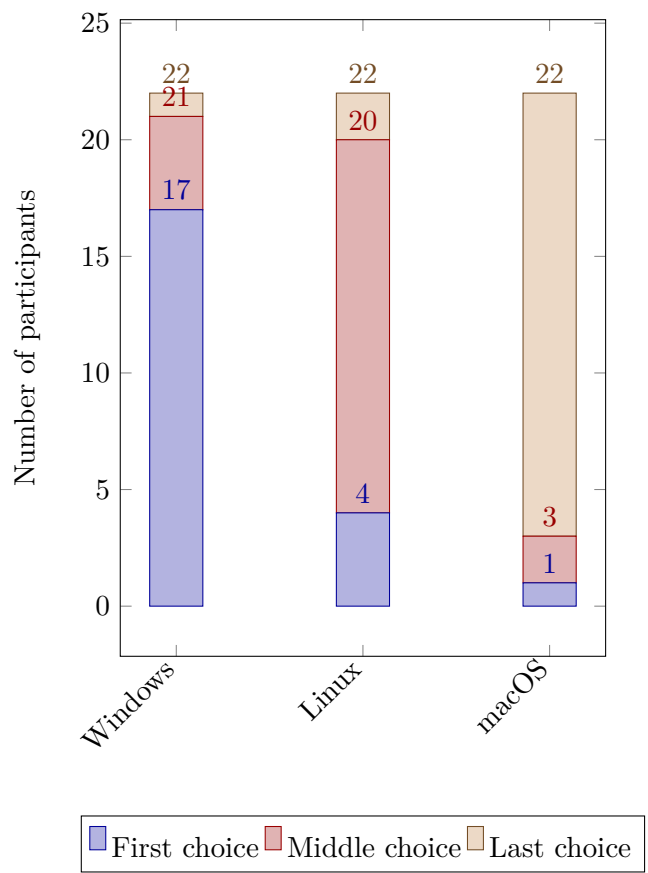


Figure 3.3: How crucial is it for the tool to support these platforms?

Would it be beneficial to have a GUI instead of only a console?

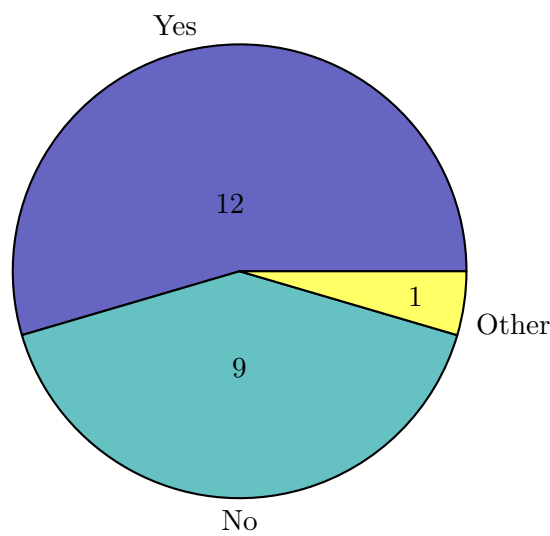


Figure 3.4: Would it be beneficial to have a GUI instead of only a console?

Other responses:

- I don't want separate docs, readable code is better.

Would you appreciate the tool being extensible via plugins?

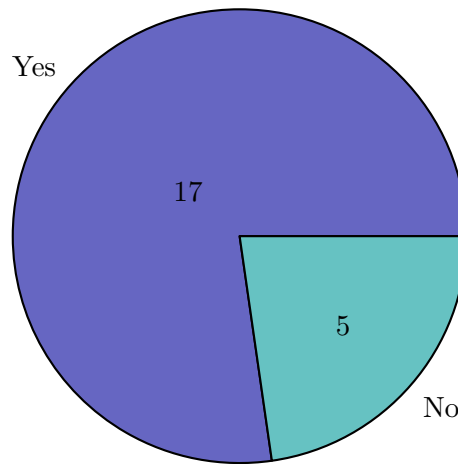


Figure 3.5: Would you appreciate the tool being extensible via plugins?

Are you using tools for documentation generation?

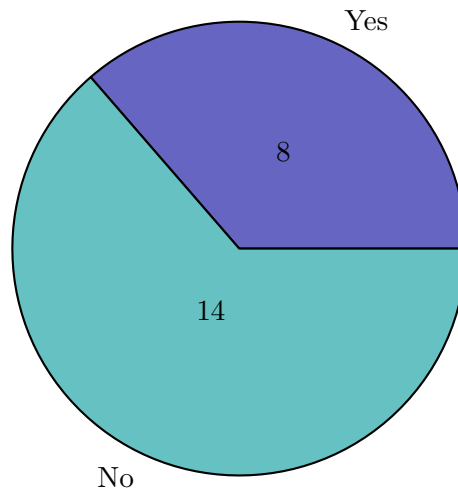


Figure 3.6: Are you using tools for documentation generation?

What documentation generating tools do you use?

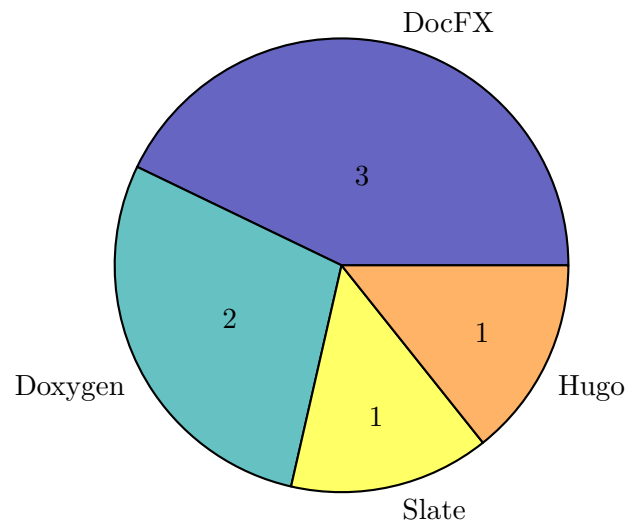


Figure 3.7: What documentation generating tools do you use?

What features from those tools you'd like to see in the resulting project?

Responses:

- GUI for less technical people.
- Links to external .NET type references.
- Easy templating.
- Extended Markdown - custom short codes.
- Fast build speed for CI/CD.
- A capable CLI.
- Class structures.

What features are missing in your current tool and would make you consider using the resulting project if they were to be provided by it?

Responses:

- Better support for diagrams (like mermaid.js).
- A tool that could build a static site with a) editorial (Markdown) content b) .NET API reference c) RESTful API documentation from ASP.NET Core projects and/or OpenAPI definitions all under one umbrella.

Would you be interested in contributing to this project?

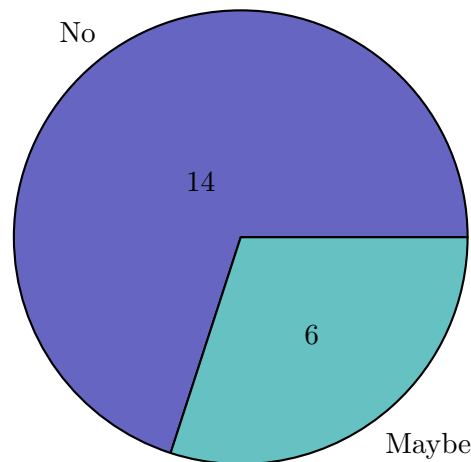


Figure 3.8: Would you be interested in contributing to this project?

3.2.2 Questionnaire evaluation

Those who find a use for such tools want to export static HTML and to have the ability to extend the functionality via plugins. Other responses displayed a clear division of needs regarding CI/CD integration, the target OS, and the need for a GUI. The dominant documentation-generating tool for .NET is DocFX (see 3.1.1).

Questionnaire takeaways:

- The majority neglects the importance of documenting code; thus, those who do and use documentation-generating tools are part of a minute subset of all developers (see figure 3.6).
- The tool should be cross-platform.
- The tool must have good performance.
- A console will be used by administrators of CI/CD pipelines and power users, while the GUI will be for the rest.
- Support for CI/CD is to automate documentation generation fully.
- Even though the majority seeks to export to static HTML pages (see figure 3.1), the project will focus on Markdown to fill the market gap. The rest of the mentioned output formats will receive support later (see below).
- **Extensibility via plugins will be the key selling point of the tool - it will enable anybody to modify the functionality to meet their specific needs.**
- Future support for DocFX (see 3.1.1) templates will help users migrate from said platform.

4. Proof of concept

Creating a proof of concept application is the ideal first step when numerous unknowns are present. Thus, investing time in this stage should reveal the potential complexity of the complete project and provide an opportunity for research, allowing further planning for the most efficient execution of development.

4.1 Milestone goals analysis

The following proof of concept analysis goals must be defined prior to development:

- What method will be used for extracting documentation?
- Where will the project be hosted?
- What is needed for outputting Markdown?

4.1.1 Documentation extraction methods

When retrieving documentation from source code, the most obvious method would be to parse said source code and extract all required data from it. Such a strategy involves writing a custom parser, probably utilizing complex regular expressions. The problem with this method is the unpredictable nature of written code, as a programmer might incorrectly format said code, the source files might not even be compilable, or a radical update to the code syntax might occur that the tool must cover. That results in unknowns that would have to be handled by the source code parser.

There might not be a need to write a custom parser, as Microsoft has already done that - the .NET compiler, Roslyn, is fully capable of parsing source code (14). Its capabilities are exposed via APIs that developers can access to create custom tools. That would be a great solution if not for the dependency on the Roslyn compiler. Unfortunately, with said dependency present, it is impossible to create a standalone tool that would work on any system, regardless of whether .NET developer tools are present.

Fortunately, reflection is also an option - the ability to examine or modify loaded assemblies during runtime (1). This feature allows developers to either reflect the running program and its references or load other non-obfuscated assemblies for reflection. This method would require correctly parsing the loaded data from the assemblies; however, unlike the previous alternatives, there is no dependency on the Roslyn compiler, nor is there uncertainty in the quality of the source of data. Given these facts, it should be possible to extract relevant data from assemblies and documentation from the XML files (see chapter 2). Of course, both data sources will have to be mapped to each other, e.g., documentation for a given class is mapped to the parsed class signature.

Thus, the proof of concept project will test the possibility of extracting all relevant information via reflection and XML files. If it proves possible, then this method could be applied in the real project. Otherwise, using Roslyn is the best alternative.

4.1.2 Open-source project hosting platforms

Jason van Gumster states in their blogpost (17) that developers have the following cloud platforms to choose from to host their open-source projects:

- GitHub.
- GitLab.
- Bitbucket.
- Azure DevOps.
- Sourceforge.

There are solutions for hosting a project locally on dedicated hardware; however, this is mainly suitable for proprietary or private projects.

As an in-depth evaluation and description of each cloud platform would reach outside the scope of this thesis, excuse the brief introduction of said platforms:

GitHub a prevalent version control hosting platform (11) with streamlined DevOps features such as issue tracking, CI/CD, dependency management, etc. GitHub was introduced in 2008, making it an established platform with over two hundred million active repositories (11).

GitLab a significant contender to GitHub (17). It provides a comprehensive suite of DevOps features and powerful CI/CD tools (17).

Bitbucket one of Atlassian's products that hosts a version control system based on Git or Mercurial (an alternative version control system) (6). The product offers public repositories for free; however, Atlassian primarily focuses on its corporate clients (21).

Azure DevOps one of Microsoft's products that hosts a version control system based on Git or TFS (an alternative version control system) (29). The product offers comprehensive DevOps features. It allows open-source projects and small teams to have access for free (29).

Sourceforge a classic platform for hosting open-source projects before Git became mainstream, declining in popularity and usage (17).

Given previous experience with GitHub and Azure DevOps, the side goal of this stage would be to assess GitLab as a potential alternative hosting platform to GitHub by publishing the proof of concept project on it. Additionally, this is an opportunity to learn how GitLab users could host the generated documentation by the custom tool.

4.1.3 Parsing data into markdown

Markdown is a simple markup language for writing formatted text. The resulting formatted text is generated from the markup by, e.g., a Git hosting platform.

Implementing support for outputting Markdown documentation for the custom tool requires the study of the markup language.

4.1.4 Development goals summary

This development stage would focus on analyzing the following questions:

- Whether it is possible to extract all relevant data from .NET assemblies via reflection?
- Whether GitLab is a viable platform for an open-source project?
- How to parse the extracted data into a human-readable output and generate Markdown?

4.2 Development results

The result of this development stage was a proof of concept console application that:

- Proved reflection as a suitable method of data extraction.
- Reminded the importance of proper design pattern use.
- Provided necessary experience with GitLab as a project hosting platform.
- Provided necessary experience with Git hosting platform's support for hosting Markdown documentation.
- Helped write a Markdown generator.
- Provided understanding of GitLab and its wiki for hosting the tools generated output.

Given these facts, it was undoubtedly a valuable investment of time and effort.

What follows are details of each analysis.

4.2.1 Reflection for data extraction

Using reflection is sufficient for extracting type and member data from selected assemblies. Nevertheless, this method requires additional effort to work correctly for the following reasons:

- The compiler generates additional types and members.
- Signatures extracted via reflection of some .NET types differ from the signatures in the source code:
 - The `dynamic` type is compiled as an `object`.

- Named value tuples (e.g., `(int age, string name)`) are presented without names as regular tuples (e.g. `Tuple<int, string>`).
- Generic type parameters are not directly accessible.
- Named value tuple names are not directly accessible.
- Determining the line number of a given member in its source code is unclear.

4.2.2 Design pattern importance

The proof of concept application was rapidly developed without intentionally applying design patterns (20). That helped to develop a simple application quickly and showed that incorrect use or absence of design patterns would produce an unmaintainable application.

4.2.3 Git platforms for hosting open-source projects

The proof of concept project source code was hosted on GitLab to test said platform's capabilities. Other platforms were not tested in the scope of this thesis.

GitLab is a freemium platform that offers all its features for public open-source projects (17). In addition, it boasts a competent CI/CD platform and a complete set of tools for Agile development. GitLab includes features for discovering other projects, but it clearly shows that it is not the platform's primary focus.

On the other hand, GitHub has the upper hand in project discoverability. It hosts many projects and has been on the market long enough to be considered a mature platform (11). Some might view the simplicity of its tools for project management as a downside. Nevertheless, if a project requires complex management tools, GitHub allows users to connect it to Azure DevOps, which has the complete tools for agile development.

4.2.4 Git platforms for hosting Markdown documentation

This stage tested four platforms for their capabilities in hosting source code documentation in their wiki pages: GitLab, GitHub, Bitbucket, and Azure DevOps.

Out of the four, GitLab has the best implementation of their wiki. It offers full support for Markdown and HTML tags. Its wiki is delivered as a separate repository, enabling users to clone and edit pages in bulk. Moreover, its content can be structured into folders for better documentation organization.

Azure DevOps is very similar to GitLab because it hosts wiki pages in a repository; thus, it can be cloned and edited in bulk. Again, the pages are separable into folders. However, the page URLs contain an additional numeric parameter unique to each page, the purpose of

which is unknown. Because of this unknown parameter and rules for its creation, supporting linking between generated documentation pages is impossible.

On the other hand, GitHub does not offer a way to structure the wiki using folders, nor does it allow users to edit pages in bulk since there is no ability to access the wiki pages as a repository. So, if the custom documentation-generating tool would generate a Markdown file for each type, then the user would have to manually upload each file to the wiki using GitHub's UI or upload the documentation with the source code.

Bitbucket has the worst support for wikis. Just like GitHub, there is no ability to edit pages in bulk. Moreover, support for Markdown is limited, as HTML tags are not rendered, headers do not have anchor links, and more. Nevertheless, this is not a problem for clients using Bitbucket, as Atlassian, the company behind Bitbucket, offers Confluence - an online knowledgebase (7).

4.2.5 Markdown generator

Creating a Markdown generator provided a deeper insight into the capabilities of the text markup language. In addition, developing the generator was straightforward and did not reveal any issues.

5. Evaluation and project planning

This milestone will focus on planning development steps and guidelines based on the evaluation of the previous milestone (see chapter 4). A concrete plan will help develop a maintainable, quality application with a clear vision of a path toward the target goal.

5.1 Evaluation

Developing the proof of concept (see chapter 4) application gave necessary insight for deciding where the project will be hosted and what technology stack it shall use.

5.1.1 Project hosting platform

The final project shall be open-source and use GitHub to host its source code. That is to allow external contributions, reach a niche target demographic, and access freemium GitHub features, which are accessible to open-source projects.

GitHub was chosen over GitLab, the second-best contender because the former platform offers better visibility and access - a crucial factor for reaching the target demographic.

5.1.2 Technology stack

The technology stack defines what technology will be used for development.

Frameworks

The proof of concept application (see chapter 4) was developed in November 2018 using .NET Framework. The resulting project will use .NET Core and .NET as it is Microsoft's latest development platform (see chapter 1). There are no reasons to stick with the old development platform.

Languages

C# will be the primary programming language, as it is the dominant language for the .NET platform (40). F# shall be used sparingly to fulfill a personal goal of exploring said language's capabilities.

DI Framework

For DI, the project shall use *Autofac*, a popular library with suitable features for this project. Choosing any other library would require investing additional time into learning how to use it; thus, no further analysis of available DI frameworks was conducted.

GUI framework

The .NET ecosystem offers a rich selection of GUI frameworks to choose from. What follows is a list of available frameworks during the time of analysis.

Available .NET GUI frameworks, summarized in a blog post (39), are grouped into ones from Microsoft and the open-source community.

From Microsoft:

- WinForms.
- WPF.
- UWP.
- Xamarin.
- Xamarin.Forms.
- Blazor.

From the open-source community:

- AvaloniaUI.
- UNO Platform.
- Eto.

The project is a developer tool for generating documentation; thus, it does not make sense to host it on the web or mobile. That leaves only desktop GUI frameworks. The only challenge is to pick one which provides cross-platform capabilities. The cross-platform targets would be Windows OS, Linux-based distributions, and macOS. Linux is the most crucial platform, as developers might want to run the tool on their CI/CD servers. It is possible that some of those servers would have a desktop interface instead of just a CLI.

At the time of the analysis, none of the frameworks provided by Microsoft qualified for the task. Furthermore, even when the company released its new framework, *.NET MAUI*, it did not include support for Linux (5).

That leaves community-built solutions. Of the three, *AvaloniaUI* was the easiest to get up and running and had a sizeable community, with extension libraries and other projects to support it. It is cross-platform and works primarily on desktop operating systems. The *UNO Platform* is very compelling on paper and is both cross-platform and cross-device (desktop, mobile, web); however, after many attempts, it was impossible even to start the demo application. *Eto*

is also a cross-platform framework but has a small community with few additional libraries.

Most .NET GUI frameworks use MVVM as their architecture, including *AvaloniaUI* (23).

Hence, the selected GUI framework for the project is *AvaloniaUI*.

5.2 Development planning

Development planning analyzes how the application will be developed and summarizes everything in the development stages (see section 5.2.5).

5.2.1 Project architecture

Applying SOLID principle helped envision an architecture of a maintainable and extendable application. The primary principle, Single responsibility (18), helps separate operations within the application into key parts (see section 5.2.2), where each part plays its vital role in the application.

The *Dependency inversion* principle (18) is applied to inch closer to an extensible application. Every critical dependency is decoupled with an interface, allowing for the *DI* design pattern (12); thus, parts of the application can be swapped and rearranged at will. This factor allows for flexible extensibility and fulfills the Open-closed principle of SOLID (18).

Implementing such an architecture would require defining said decoupled interfaces first. The next step would be to create specific implementations of said interfaces. E.g., *Diagram Generator* would be the interface, while *PlatUML Diagram generator* or *MermaidJS Diagram generator* would be the specific implementations deriving from said interface.

Specific implementations of an interface will be referred to as **components**. Given that there might be various implementations of one interface, a group of specific implementations would be referred to as a **plugin**. A user would install only those plugins that they need for their task. Moreover, a developer would create a custom plugin by grouping specific components that already exist or they would have written themselves.

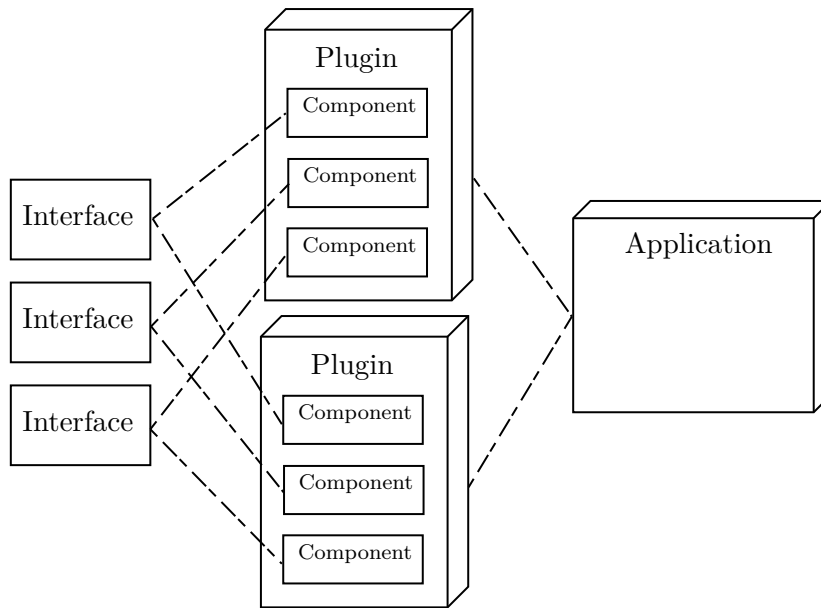


Figure 5.1: Application architecture

5.2.2 Interfaces

The primary principle, Single responsibility (18), separated operations within the application into the following key parts:

- Member provider.
- Documentation provider.
- Composer.
- Element provider.
- Diagram generator.
- Linker.
- Printer.

Each such part will be represented using an interface, as described above (see section 5.2.1).

Member provider

The member provider will be responsible for loading type and member data from a given collection of sources into a collection of a defined data structure. This collection will then be provided to the consumer of this key part via the relevant interface. A concrete interface implementation would define the source type and the method for processing the given source.

Documentation provider

The documentation provider will be responsible for loading documentation for specific types and members from a given collection of sources, and mapping said documentation to the relevant types and members. The map will then be available for querying to consumers of this key part via the relevant interface. A concrete interface implementation would define the source type and the method for processing given source.

Element provider

The element provider will be responsible for providing structural elements for the visual representation of the generated documentation. Such elements would include:

Text regular text without any formatting

Formatted text text with formatting, e.g., bold, italic, code

Lists numbered, bullet, term lists

Tables headered tables

Sections content separators with headings containing all of the above

Pages content separators containing sections

Consumers of this provider would create the required elements via the relevant interface. A concrete implementation of the interface would define how said elements are to be represented, e.g. Markdown, HTML, $\text{\LaTeX}$.

Diagram generator

The diagram generator will primarily create inheritance diagrams for the provided types. Additional diagram features, such as a class structure, relational database models, etc. won't be part of this thesis. A concrete interface implementation would define how said diagrams will be delivered, e.g., as text or as a link to a file. In the case of text, they could be delivered as ASCII art, PlantUML or MermaidJS diagrams.

Linker

The linker interface will create links between resolved types and anchor links to sections of documentation pages. A concrete implementation of this interface would define how said links are to be generated to fit the target platform, e.g., GitHub and GitLab have different link formats.

Composer

The composer interface will iterate through the resolved data, combining and filtering it, then composing it into section and page elements from the respective provider (see ‘Element provider’ on page 44). Specific implementations of this interface would define the result of the composition.

Printer

The printer interface will export the composed elements (see ‘Composer’ on page 45) into the desired format. Specific implementations will define how and where the result is exported.

5.2.3 GUI application

Based on the developer needs analysis (see section 3.2), users of such a tool expect a CLI application. However, configuring such an extensible tool via the command line can be overwhelming. That is solved by providing a GUI application for creating configurations, testing them, and later creating a CLI for executing the generated configurations in, for example, a CI/CD pipeline. To summarize, the purpose of this application would be to provide.

5.2.4 Plugin structure

As described in ‘Project architecture’ on page 42, plugins will combine specific components that define the plugin’s functionality. Since components are supposed to be independent, they must be individually configured. Thus, the plugin must provide configuration steps for the application to consume and display to the user. The internal architecture of the plugin is up to its developer; the only mandatory conditions are:

- Provide configuration steps for the components.
- Provide an endpoint for executing the configured components.

5.2.5 Development stages

Since the project is divided into interfaces, components, plugins, and the UI application, it makes sense to define development steps based on said architecture.

1. Define the shared interfaces
2. Create the reflection-based member resolver component
3. Create the XML documentation resolver component
4. Create the Git link resolver component

5. Create the Markdown elements component
6. Create the MermaidJS diagram generator component
7. Create the composer component
8. Create the printer component
9. Create the GUI application
10. Create the Markdown plugin

Completing all these steps will be the solution to the goal of this thesis. For simplicity, said libraries will be developed specifically for processing .NET projects written in C#.

5.3 Project name

The last crucial decision is choosing a name for the custom documentation-generating tool, as referring to it using said description is impractical at this project stage. As this is an open-source project for developers, there are few to no constraints on the project name. Nevertheless, choosing something easy to read, pronounce, and understand while describing the project is still a good idea. This requirement helps prevent naming the project using abbreviations, complex words, or words that have nothing to do with the project.

Thus, the developed custom documentation generating tool will be called **ModularDoc** because this name is:

- Descriptive.
- Easy to read.
- Easy to pronounce.

6. Processing libraries

This milestone summarizes the development results of stages 2 - 8 described in ‘Development stages’ on page 45.

6.1 Reflection-based member provider

The reflection-based member provider is the most important library of this project, as it shall provide type and member information, which will then be mapped to extracted documentation. If fully implementing this library using reflection fails, the project’s complexity would skyrocket, as the only viable option is parsing source code directly.

The proof of concept (see chapter 4) was built using .NET Framework and utilized the built-in reflection capabilities. Moreover, all tests were done on non-complex assemblies without references. That was a fatal oversight. .NET Core provides reflection capabilities; however, it does not allow loading an assembly without its references. That meant that the researched solution was useless, and an alternative had to be found.

Fortunately, after considerable effort, it was. The issue was solved by a third-party reflection library called *dnlib*, which is most notably used for obfuscating code in .NET assemblies using its assembly modification features. For the purposes of reading and extracting required data, *dnlib* was more than sufficient.

6.1.1 Separation of types

ModularDoc would allow the selection of individual assemblies. That implies that a user can select only a subset of assemblies generated by a project; thus, introducing the possibility of exclusion of a referenced assembly and its types.

The member resolver would produce a set of types defined within the given assembly and types referenced by their members. Once all assemblies are loaded and the following process accesses a referenced type, it can verify whether the type is provided via the loaded assemblies. If it is not, only the type name is preserved.

That is especially important for the Linker component, which will create links between references and documentation of said types.

6.1.2 Dynamic types and value types

Both *dnlib* and the built-in .NET reflection do not provide complete details about the extracted types and members. That implies that additional effort is needed to extract the necessary information. In some cases, such an attempt is much more considerable than in others.

The most complex cases were for members that either returned or accepted `dynamic` types and value tuples. A `dynamic` type is a regular `object` type but with compile-time syntax checking disabled. That allows developers to access type members without casting it to their corresponding type. A value tuple is an extension of the regular C# tuple but with more syntactic sugar that allows labeling each tuple entry with a name.

Listing 6.1: C# tuple and value tuple examples

```
1      Tuple<int, string> studentAgeClassic; // Classic tuple
2
3      (int age, string name) studentAgeNew; // Value tuple with labels
4      (int, string) studentAgeNewNameless; // Value tuple without labels
```

Reflection displays `dynamic` types as `object` types and hides value tuple labels. That obfuscates the edge cases mentioned above.

Necessary information for deobfuscating said cases is hidden in attributes decorating the given member. Said attributes provide a flat map indicating which types are `dynamic` and value tuple labels. However, this result cannot be used straight away since returned or accepted types by a member can be more complex. For example, the type can be a tuple of objects, some of which are `dynamic`. Alternatively, the type could be a value tuple with one of its entries also being a value tuple. That introduces structural complexity that cannot be mapped trivially to a flat list provided by the attributes.

The solution for this is a complex algorithm for processing the flat list from the attributes, filtering out unnecessary information, and mapping it to the provided type from the member.

6.1.3 Missing features

The developed implementation of the member resolver component covers many cases and provides plenty of helpful information about types from given assemblies. Unfortunately, only some things were implemented, either because it was not a priority or needed to be more apparent on how to implement it.

Attributes

Attributes are annotations for types, members, or parameters that add metadata. ModularDoc can generate documentation for attributes, as they are just classes inheriting a particular

`Attribute` class, but it does not link attributes to types, members, or parameters they decorate.

Links to source code

It can be beneficial to quickly access a line of code defining a given documented type or member. For example, the documentation is hosted on a Git platform and links to the source code's exact lines. Nevertheless, it was impossible to create a working solution, as some assemblies provided the necessary information for unknown reasons while others did not. Thus, this feature cannot be delivered, as it is unstable.

Reference to source assembly

It can be beneficial to see from which assembly the given types were extracted. It is trivial to implement, although this feature had a low priority and was left out of this development cycle.

Multicast delegates

Regular C# delegates are defined within a class, whereas multicast delegates are defined on the same level as a class. The existence of multicast delegates was, unfortunately, overlooked when developing the tool. Such delegates are compiled as classes deriving from the `MulticastDelegate` base class. That means that they are captured but are incorrectly represented as classes.

6.2 XML documentation provider

Writing the XML documentation processing component involved:

- Analyzing Microsoft's documentation of supported tags.
- Defining the corresponding data structures to hold the extracted documentation.
- Attempting to implement more complex issues like locating inherited documentation.

6.2.1 Tags documentation

Microsoft's documentation for some tags, such as the list tag (see section 2.2.3), could be more specific. Such tags have missing examples and details that fully define how to use them. Moreover, Microsoft's IDE, Visual Studio, does not display some of the tags (including the list tag). Thus, research outside Microsoft's documentation unnecessarily slowed defining logic for handling such tags.

6.2.2 Dependency on the member provider

Because of the documentation inheritance tag (see section 2.2.4), the documentation provider must be capable of locating the source of inheritance. This information is not necessarily present in the source XML, as specifying the inheritance source is optional and only needed if more than one source provides unique documentation for a single inherited member.

Thus, the documentation resolver had to have access to retrieved data from the member provider. Inherited documentation would be lazily loaded once needed. That is to make the documentation provider and member provider work in parallel instead of waiting for one another. This lazy-loading technique is utilized throughout ModularDoc to improve the parallelization of processes dependent on each other.

6.3 Linker

The linker component was one of the projects chosen to be developed using F#. It was chosen because the linker does not need any data structures and generates an output based on the processed input.

6.3.1 Git-specific logic

This component is focused on providing link and anchor formats for a select number of Git hosting platforms. It would support four platforms if not for issues with Bitbucket and Azure DevOps. Instead, only GitHub and GitLab are supported.

6.4 Markdown elements

The markdown elements provider component was researched in the proof of concept stage (see chapter 4), and the gained knowledge had proven sufficient for implementing this component.

6.5 MermaidJS diagram generator

The diagram component can generate images and provide links to them or generate text-based diagrams. *MermaidJS* is a diagramming tool written in JavaScript (27). The tool receives Markdown-inspired text definitions and converts them into diagrams (27). An alternative to this diagramming tool is *PlantUML*. However, GitHub supports only *MermaidJS* for diagram generation (16), making it the obvious choice for the diagramming component.

6.6 Composer

The composer component is one of the most complicated, as it maps the extracted data to each other and fills it into the provided elements. This component was developed in F# for the same reasons as the linker (see section 6.3). Whether or not that was a mistake is still up for debate; nevertheless, the resulting code is complex and difficult to maintain.

6.6.1 Composed structure

The composer is responsible for creating the documentation structure. The defined structure was inspired by the one generated by Doxygen (see section 3.1.2):

- Inheritance diagram.
- Table of contents for Methods and Properties.
- Full documentation.

Inheritance diagram

A diagram-generating component provides the inheritance diagram content. The purpose is to see what interfaces and base classes are inherited by the given documented type. Additionally, the diagram would display what generic parameters each type in the inheritance tree has alongside their constraints.

Table of contents

The table of contents briefly overviews the type's method and property members. The overview contains the members' names and summaries. In addition, each documented entry in the table of contents is linked to its complete documentation (see below).

Full documentation

The rest of the generated documentation for a given type covers all extracted data in great detail. It is sectioned as follows:

- Written documentation for the type - documentation extracted from the XML.
- Type inheritance - a flat list of all types inherited by the documented type.
- Member documentation - each member documented based on extracted data from XML.

6.7 Printer

The printer component was the simplest to implement, as it only processes the composer-generated output and writes it into Markdown files.

7. GUI application

This milestone covers the development effort for implementing stage 9 described in ‘Development stages’ on page 45.

7.1 AvaloniaUI

As decided in the project planning milestone (see chapter 5), *AvaloniaUI* will be used as the GUI framework. Because *AvaloniaUI* uses XAML for defining the UI and utilizes MVVM as its architectural pattern (23), it was not too difficult to use it thanks to previous experience with WPF (one of Microsoft’s official GUI frameworks).

The only drawback of the framework is that it still needs a stable release. That entails the unreliability of the framework and the possibility of breaking changes. For example, *AvaloniaUI* version 0.10.15 broke UI components for displaying images. Nevertheless, it is backed up by a passionate community of developers that ensure a quick resolution of any discovered issues.

The open-source community added significant value to the framework by developing additional libraries such as *AvaloniaBehaviors*, or *FluentAvaloniaUI*, both of which are used by ModularDoc. The former extends missing features for manipulating the UI based on defined triggers. The latter provides the *Fluent* theme for *AvaloniaUI*. *Fluent* is Microsoft’s modern design system that can be seen utilized across all of its applications (30). By default, the *AvaloniaUI* uses an outdated version of *Fluent* and, in some cases, does not respect the design system. So, the *FluentAvaloniaUI* library is essential for having a modern, good-looking design.

7.2 Decoupling via MVVM

AvaloniaUI is a beautiful GUI framework; however, it is not a first-party Microsoft product - this poses a higher risk of project abandonment. Moreover, users of the ModularDoc might not like *AvaloniaUI* and wish to have the tool hosted online in an ASP.NET Core web application. Doubling down on a highly configurable tool results in an application with its business logic utterly independent from the UI framework. That is achieved using MVVM in combination with a DI framework like *Autofac*.

MVVM (Model, View, View Model) allows natural decoupling of the UI (View), from the business logic (View Model) and data (Model) (23). However, MVVM alone still introduces some coupling of the UI to the host application, as it must still reference the specific views for, e.g., navigation. Such references are decoupled by abstracting each view with its interface

representation. The specific views would then be provided via *Autofac*. This allows easy swapping of views either based on *AvaloniaUI* or some other framework.

To take it a step further, the view models are also abstracted in the same way and provided via *Autofac*. In this case, the benefit is higher application testability, as abstracted view models can be mocked.

7.3 Application structure

The GUI application serves only as a configuration creator and executor for the available plugins. Thus, the application requires minimum work on the UI side and will consist of the following pages:

- Home page.
- Settings page.
- About page.
- Configuration wizard.
- Summary page.

Each page was mocked to understand the application's UI layout before its implementation.

7.3.1 Home page

The home page presents the user with the available plugins. The UX is simple, as the user should only select the desired plugin, see its description, and decide whether they wish to use it or cancel. Additionally, the user may wish to load a previously created configuration, so an option for loading configurations is available. Finally, the user can search for the desired plugin via the search filter if the application has many of them.

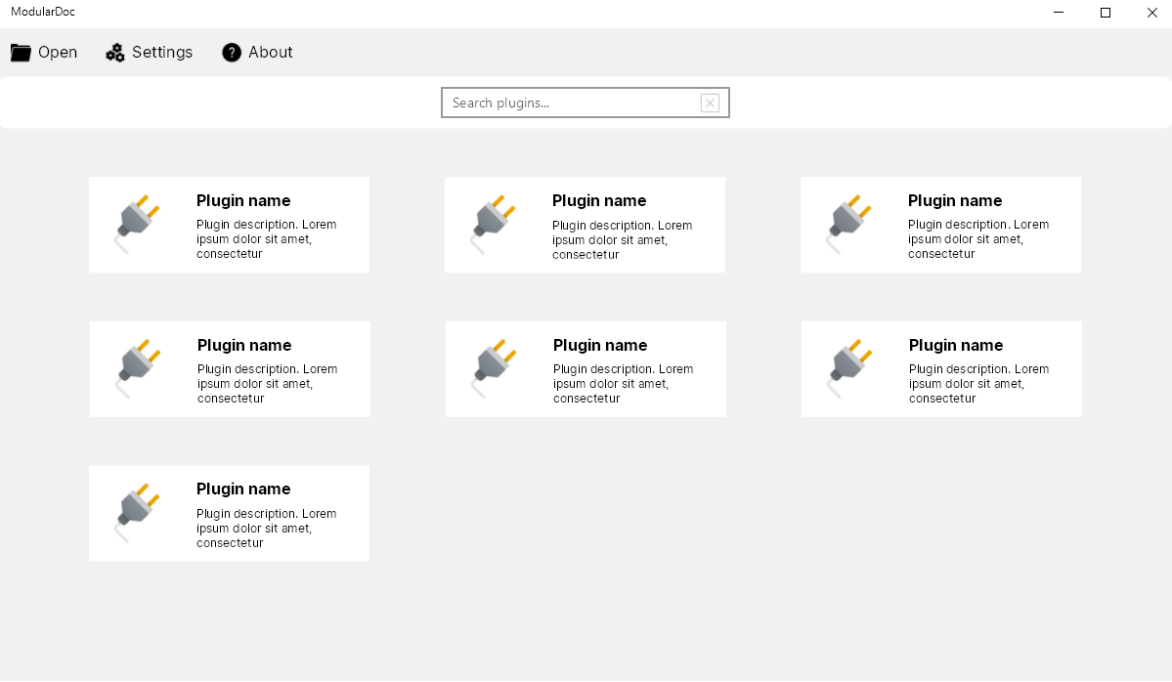


Figure 7.1: Mock of the home page

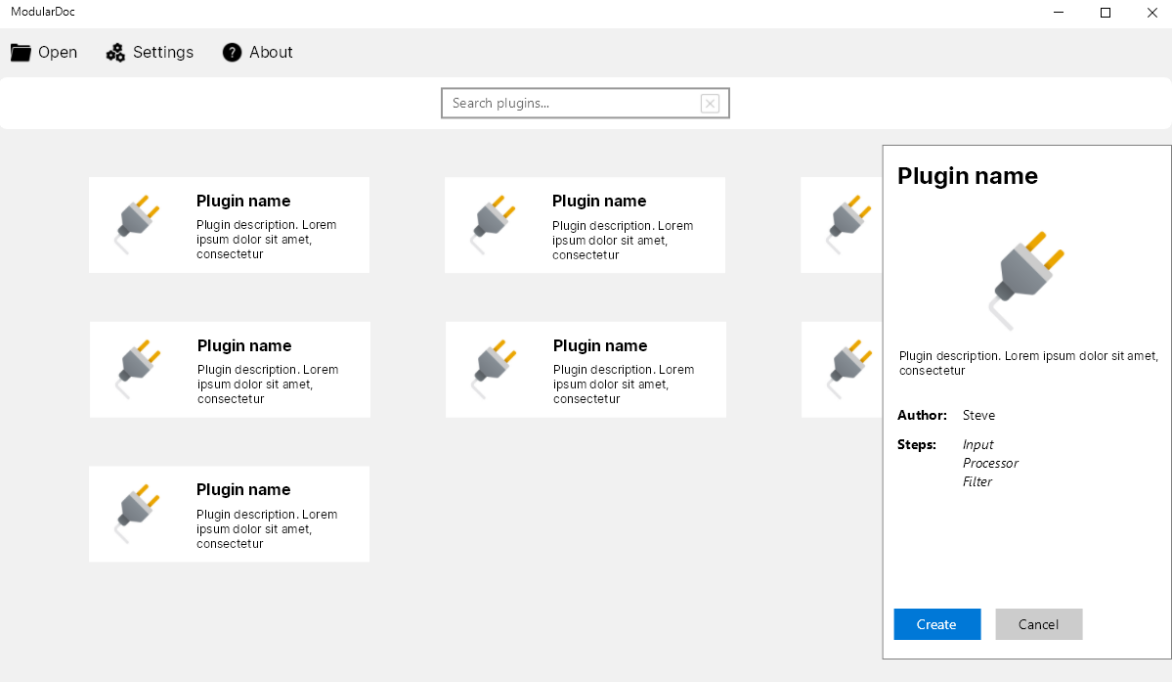


Figure 7.2: Mock of the home page with a plugin selected

7.3.2 Settings page

The settings page allows the user to configure the application. Since adding multicultural support would add more complexity to the project, the only setting provided on this page is a toggle between the light and dark mode themes. Any settings added in the future will be present on this page.

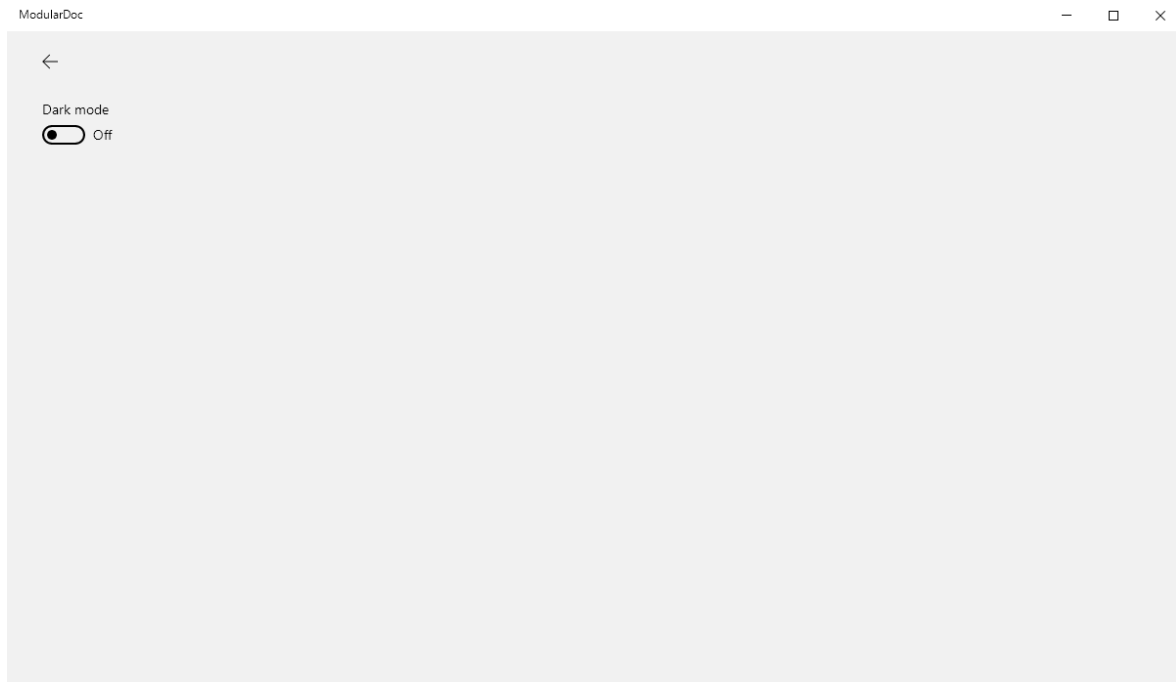


Figure 7.3: Mock of settings page

7.3.3 About page

The about page would present the user with necessary information about the application, references to used libraries, and possibly usage instructions. However, the content of this page was neither designed nor prepared. That is because it had a low priority; thus, it was excluded from this development phase.

7.3.4 Configuration wizard

The configuration wizard is responsible for displaying configuration steps provided by the selected plugin. The plugin provides the UI of each step; thus, this page's design is only concerned with correctly displaying the steps and controls for navigating between them.

The user can proceed to the next step only if the current one is configured correctly.

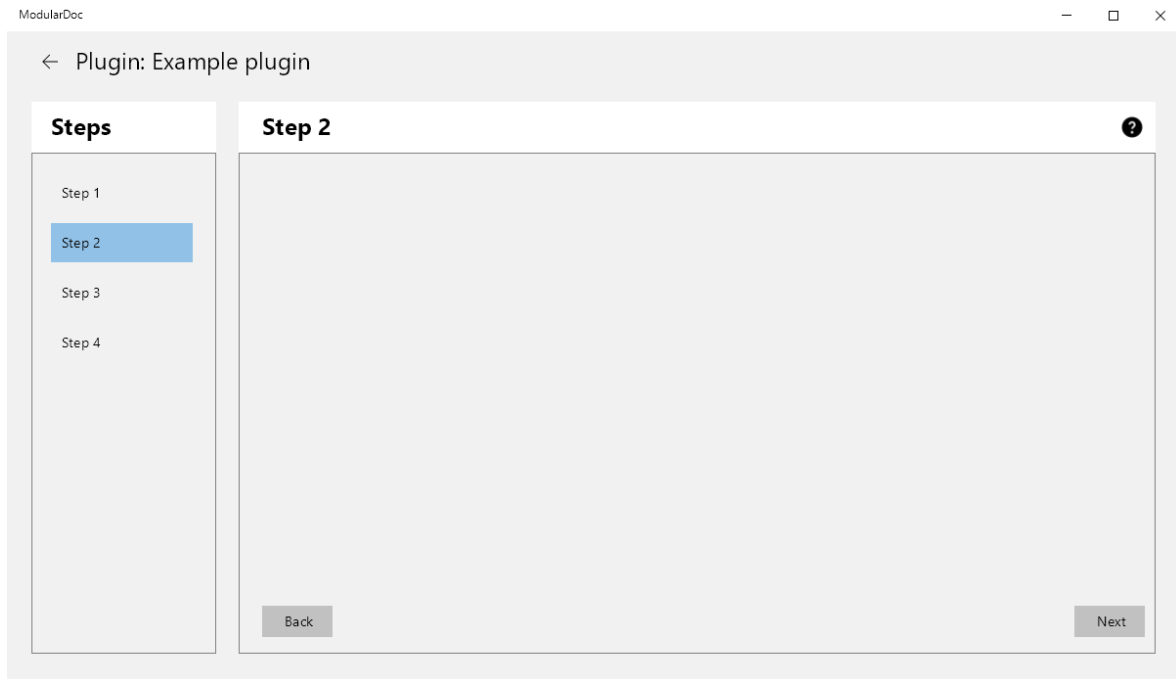


Figure 7.4: Mock of configuration wizard

7.3.5 Summary page

The summary is responsible for executing the selected plugin’s configuration after loading it or configuring it for the first time. In addition, it displays the progress of each plugin step and logs what each process is doing.

At this stage, the user can save the created configuration and return either to the first configuration step or to the home page without saving.

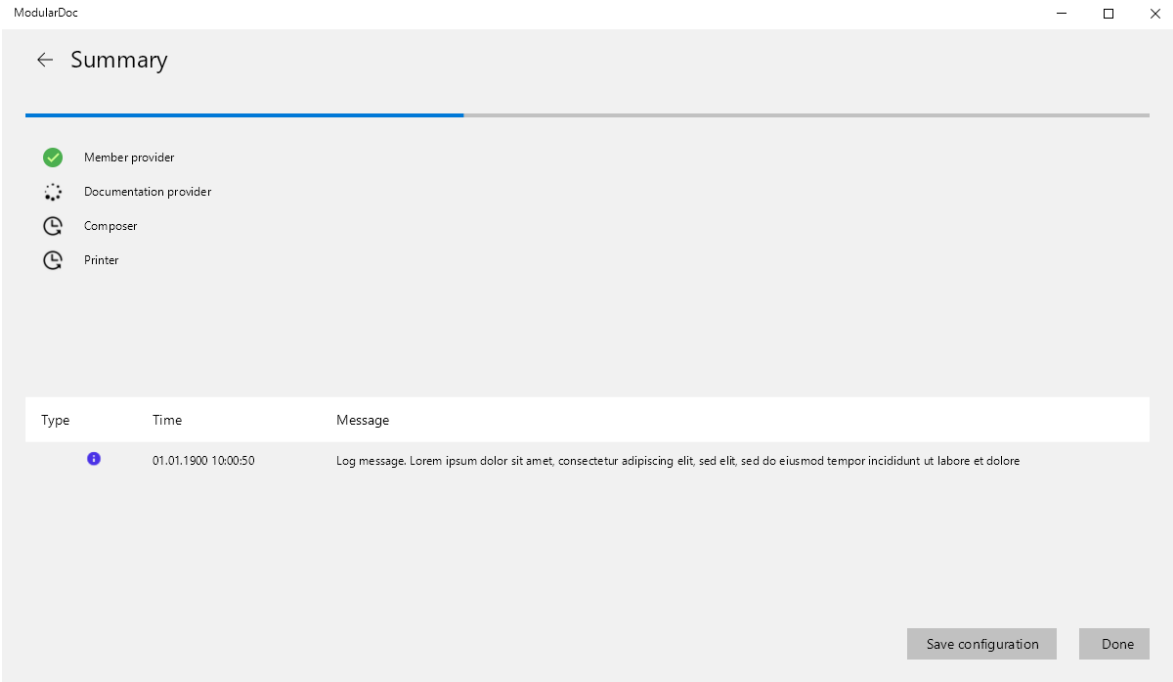


Figure 7.5: Mock of summary page

8. Markdown for Git plugin

This milestone covers the development of the Markdown for Git plugin described in stage 10 of the ‘Development stages’ on page 45.

8.1 Plugin structure

The Markdown for Git plugin will be composed of components developed in ‘Processing libraries’ on page 47.

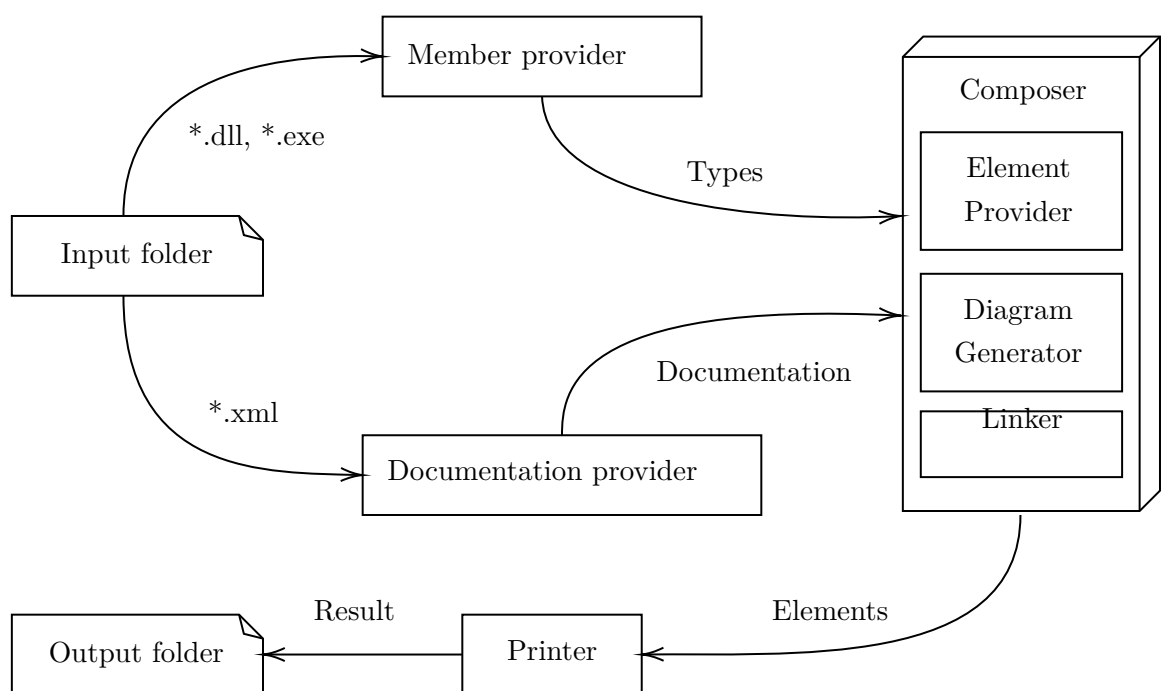


Figure 8.1: Plugin components structure

Figure 8.1 depicts the relations between each component, input folder, and output folder. First, the member and documentation providers operate in parallel and supply necessary data to the composer component by processing files from the input folder. Next, the composer component depends on the element provider, linker, and diagram generator for composing the extracted data into a structured documentation. Finally, the structured documentation is fed into the printer component, which generates the physical files into the designated output folder.

8.2 Plugin configuration steps

The plugin must provide configuration steps for the user to set how each component should operate. The plugin does not need configuration steps for every component, as some might not be configurable. Additionally, the plugin might provide configurations that affect more than one component.

In the case of the Markdown for Git plugin, the following configuration steps are necessary in this order precisely:

1. Member provider.
2. Documentation provider.
3. Linker.
4. Global configuration.

8.2.1 Member provider

The member provider configuration would be labeled to the user as the **Assembly** processing component because they would select the desired paths to assemblies and executable .NET files for processing. The selection process can be performed in the following ways:

- By providing the path in the text input manually and pressing the add button.
- By pressing the search button and selecting the desired paths via the file browser
- By pressing the add button when the text input is empty and selecting the desired paths via the file browser

The user can move to the next step once at least one path is provided. Individual paths can be removed if needed. Each path has the filename highlighted for improved readability.

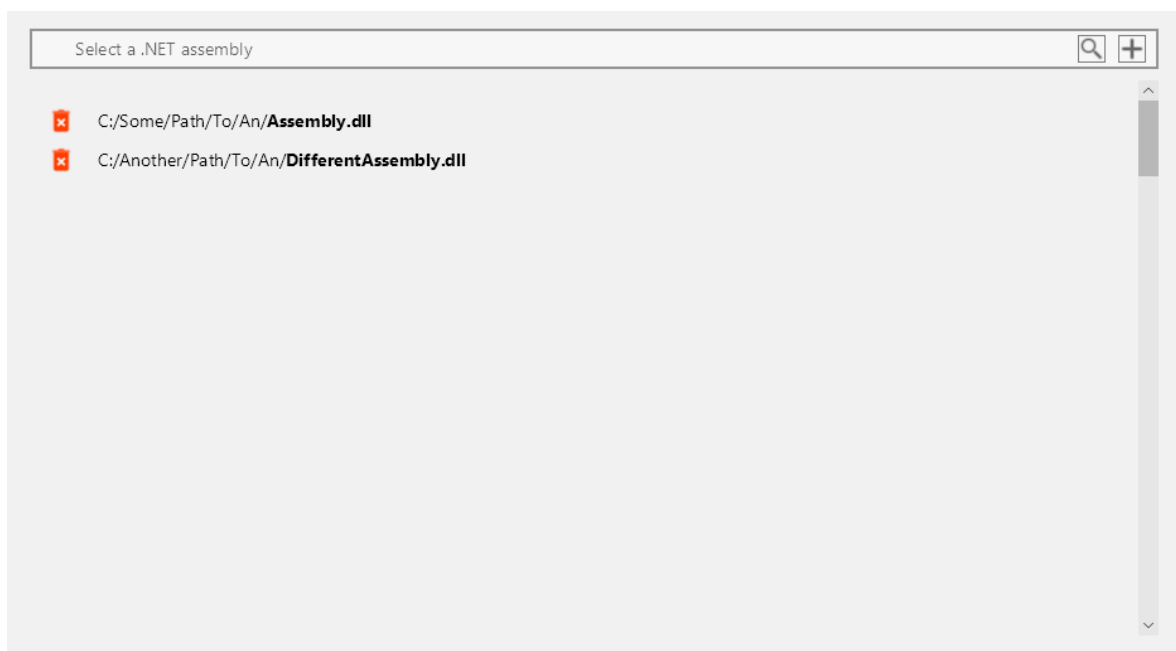


Figure 8.2: Mock of the assembly configuration step

8.2.2 Documentation provider

The documentation provider step displays the automatically discovered documentation files alongside those missing. The initial vision was to allow the user to resolve the missing files by ignoring their absence or locating them on their device. However, this was not implemented due to limited development time, and the missing paths are simply displayed without the options for ignoring and finding.

All paths have the filename highlighted for improved readability.

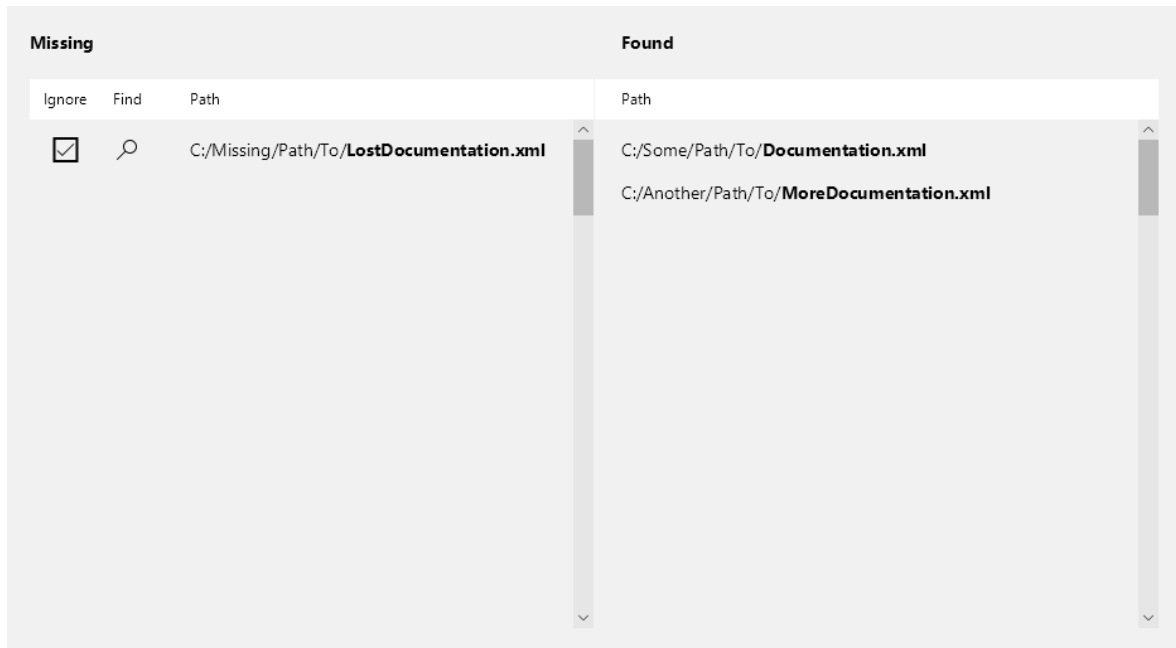


Figure 8.3: Mock of the documentation configuration step

8.2.3 Linker

The linker step displays configuration options that target specific Git platforms. The user can generate documentation either for GitHub or GitLab. The choice affects the generated link format, which is platform-specific.

Next, they can decide whether the documentation will be hosted in the repository alongside the source code or the wiki pages. This option is necessary for GitHub since its wiki limits its users to a flat file structure, whereas GitLab allows structuring documentation into folders in its wiki. Moreover, GitHub does not allow uploading multiple pages to the wiki in bulk. Thus, if there is much documentation, a user would most likely need to use the option for hosting documentation alongside the source code.

Finally, it is possible to explicitly choose whether the documentation is generated flat or structured into individual folders. This option is relevant for GitLab, as it allows both.

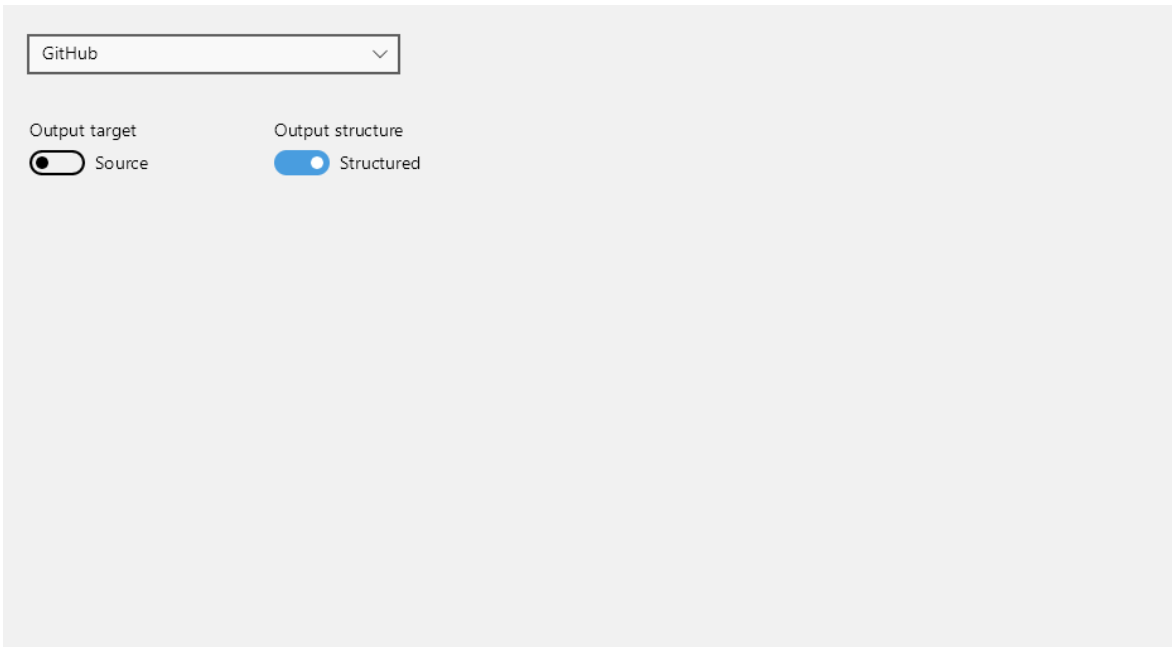
A mockup of a configuration interface for a linker step. At the top, there is a dropdown menu with 'GitHub' selected and a downward arrow. Below this, there are two toggle switches. The first is labeled 'Output target' and has two options: 'Source' (selected with a black circle) and 'Wiki' (unselected). The second is labeled 'Output structure' and has two options: 'Flat' (unselected) and 'Structured' (selected with a blue circle).

Figure 8.4: Mock of the linker configuration step

8.2.4 Global configuration

The global configuration affects all components. It provides a filter for excluding specific namespaces and types from being processed. Additionally, it allows users to specify the output folder for the generated documentation.

Namespaces and types are ignored by unchecking the nodes representing them. If a given node with children (e.g., a namespace) is unchecked, its children will also be ignored. Types are italicized for improved readability.

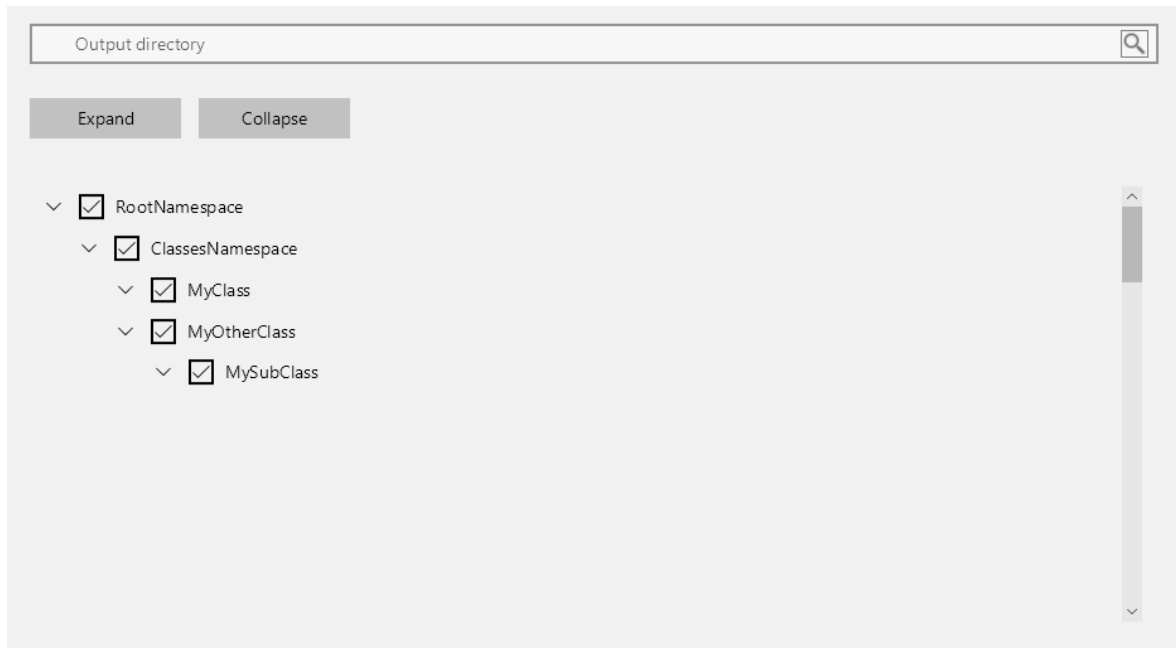


Figure 8.5: Mock of the global configuration step

9. Performance

One of the goals of this thesis was to ensure that the resulting application performance is comparable, if not better, to existing tools. Thus, it is necessary to evaluate whether this requirement is satisfied.

The thesis project source code would be used for benchmarking performance. Specifically, projects containing the shared interfaces shall be used as the source of processing¹. The tests will run on a Windows OS and shall compare the performance of ModularDoc to DocFX, and Doxygen. Source browser was excluded due to unclear instructions and the inability to make it work using existing examples. Other solutions were also excluded as they do not produce proper documentation.

Testing will measure elapsed time for generating the documentation. This will be done by running each tool via Windows PowerShell using the *Measure-Command*. If the resulting tool is consistently perceived to be considerably slower than existing tools, then the performance requirement can be deemed unsatisfactory.

9.1 Configurations

Each tool required configuration to work correctly. What follows is a description of how said tools were configured.

9.1.1 ModularDoc

The performed benchmarking is relevant for the Markdown for Git plugin, as it is currently the only available plugin for the tool. For consistency, a console-based application was created that loads the defined configuration and executes it. The console application is missing crucial features such as argument validation and error handling; thus, it primarily serves testing purposes.

The plugin was configured to output structured documentation for GitHub; however, modifications to the configuration affected performance insignificantly.

9.1.2 Doxygen

Configuring Doxygen was simple and intuitive, making it a worthy contender. The tool was configured to:

¹Located in the solution folder, *Libraries/Core/*. Everything except the *Constants* project.

- Document all entities and include those without documentation.
- Optimize results for Java or C# output.
- Generate plain HTML output with search capabilities.
- Use the built-in class diagram generator.

Version 1.9.5 was used for testing.

9.1.3 DocFX

Configuring DocFX could have been more intuitive, as everything is done via the CLI. However, the documentation was sufficient for configuring the tool to process only the desired input.

The tool was configured by modifying the *docfx.json* in a DocFX project to use ModularDoc's source code as its input for metadata.

Version 2.59.4 was used for testing.

9.2 Results

Run	ModularDoc	Doxygen	DocFX
1	864 ms	3609 ms	34268 ms
2	968 ms	1784 ms	28662 ms
3	869 ms	1664 ms	28681 ms
4	846 ms	1660 ms	28681 ms
Average	<i>886.75 ms</i>	<i>2179.25 ms</i>	<i>30022.25 ms</i>

Table 9.1: Documentation generating tools performance comparison

As clearly shown, ModularDoc is at least ten times faster than Doxygen and a hundred times faster than DocFX. That is likely because the ModularDoc is designed explicitly for .NET projects, whereas the other two are polyglots that support different languages and project types. Additionally, the tool was designed with performance in mind. ModularDoc utilizes caching techniques, lazy loading, and, most importantly, uses reflection instead of parsing source code.

10. Tool usage and extension guides

Consult this chapter to learn how to use and extend of ModularDoc.

In the context of this thesis, ModularDoc was developed with only one plugin - Markdown for Git. Thus, the user guide covers the tool with this plugin in mind.

10.1 User guide

ModularDoc is delivered as a portable application without an installer. Thus, mentioned files and folders are in the context of the ones provided with this thesis (see ‘Appendix: ModularDoc compiled’ on page 87) or compiled from the source code (see figure 10.1). The source code is also provided with this thesis (see ‘Appendix: ModularDoc source code’ on page 87); however, the most up-to-date version can be found on GitHub (see ‘Open-source’ on page 81).

```
ModularDocInstallationRoot/  
├─ ModularDoc.App.exe  
├─ Components/  
├─ Libraries/  
├─ Plugins/  
│   └─ GitMarkdown/  
│       └─ ModularDoc.Plugins.GitMarkdown.dll  
│       └─ ...  
└─ ...
```

Figure 10.1: ModularDoc simplified file and folder structure

10.1.1 Prerequisites

- A desktop device running either of the following OS:
 - Windows 7+
 - macOS High Sierra 10.13+¹
 - Debian 9 (Stretch)+, Fedora 30+, or Ubuntu 16.04+ (Linux distribution with *glibc* 2.17 installed, or a newer version)²
- The OS must be 64bit
- .NET Runtime 6.0.11+ is installed
- Free 270 MB of disk space

¹ModularDoc was not tested on a macOS device, due to lack of Apple hardware

²ModularDoc was tested on PopOS 22.04 LTS, a Linux distribution from system76

10.1.2 ModularDoc application

The ModularDoc application can be started by running the *ModularDoc.App.exe* file (see figure 10.1).

Home page

The user can select one of the available plugins within the application and start configuring it via the configuration stepper, then run and save the configuration on the summary page. Additionally, the user can load the created configuration, edit it or run it.

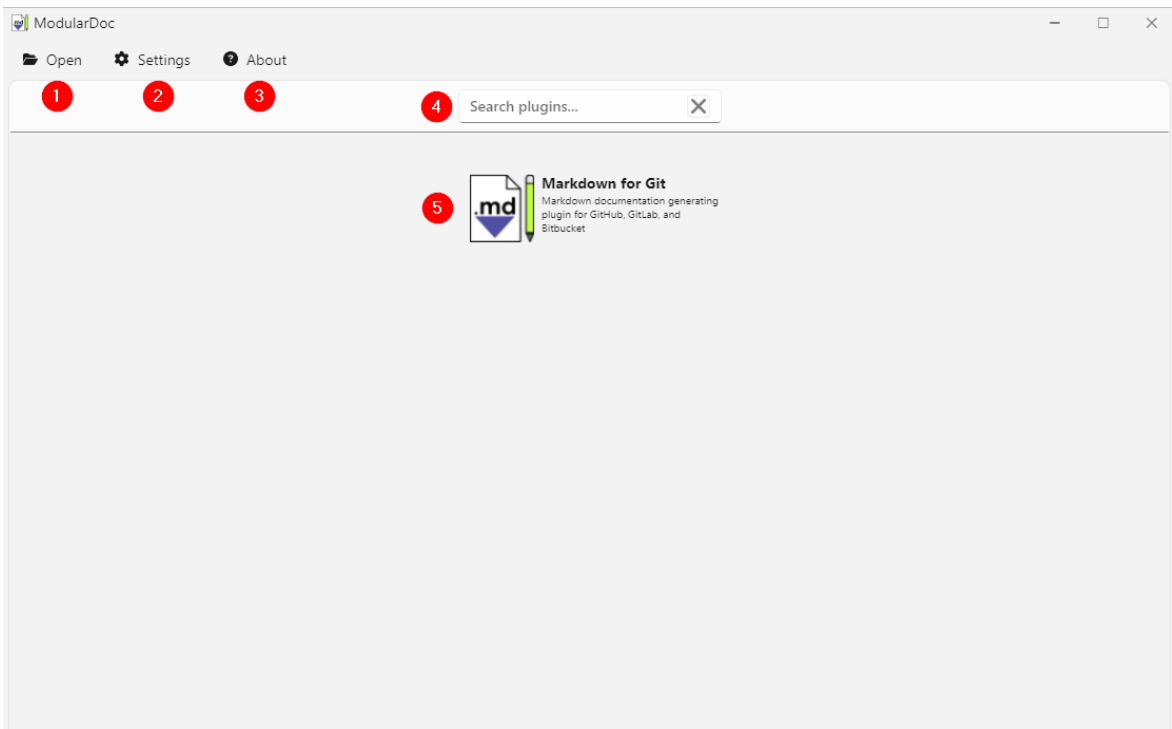


Figure 10.2: *ModularDoc* home page

1. Open button for loading previously defined configurations
2. Settings button for navigating to the settings page
3. About button for displaying information about the application (not implemented)
4. Search entry for filtering the available plugins
5. One of the available plugins with its icon, title, and short description

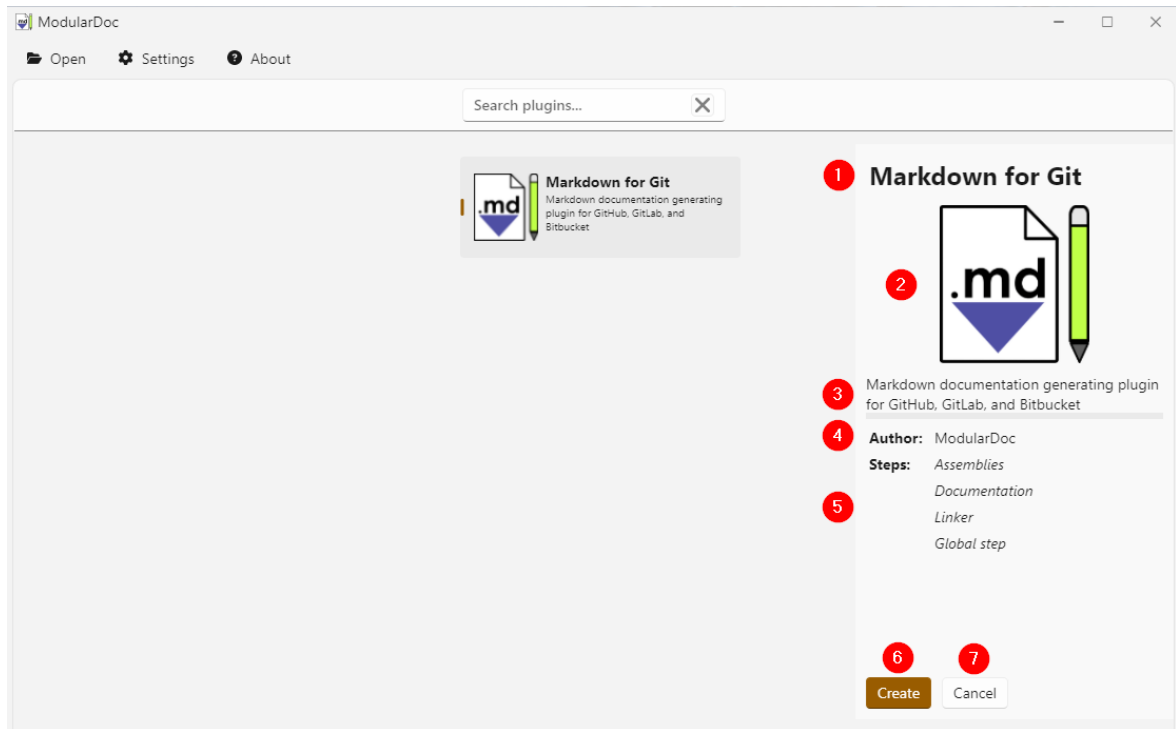


Figure 10.3: ModularDoc home page with a plugin selected

1. Plugin title
2. Plugin icon
3. Plugin description
4. Author information
5. Steps within said plugin
6. Button for creating a configuration using the selected plugin
7. Button for closing the selected plugin panel

Configuration wizard

The configuration wizard presents the selected plugin's steps. Each step's settings are displayed for configuration. The user can view additional help for each active step. Navigating to the next step is possible only if the active one is configured correctly.

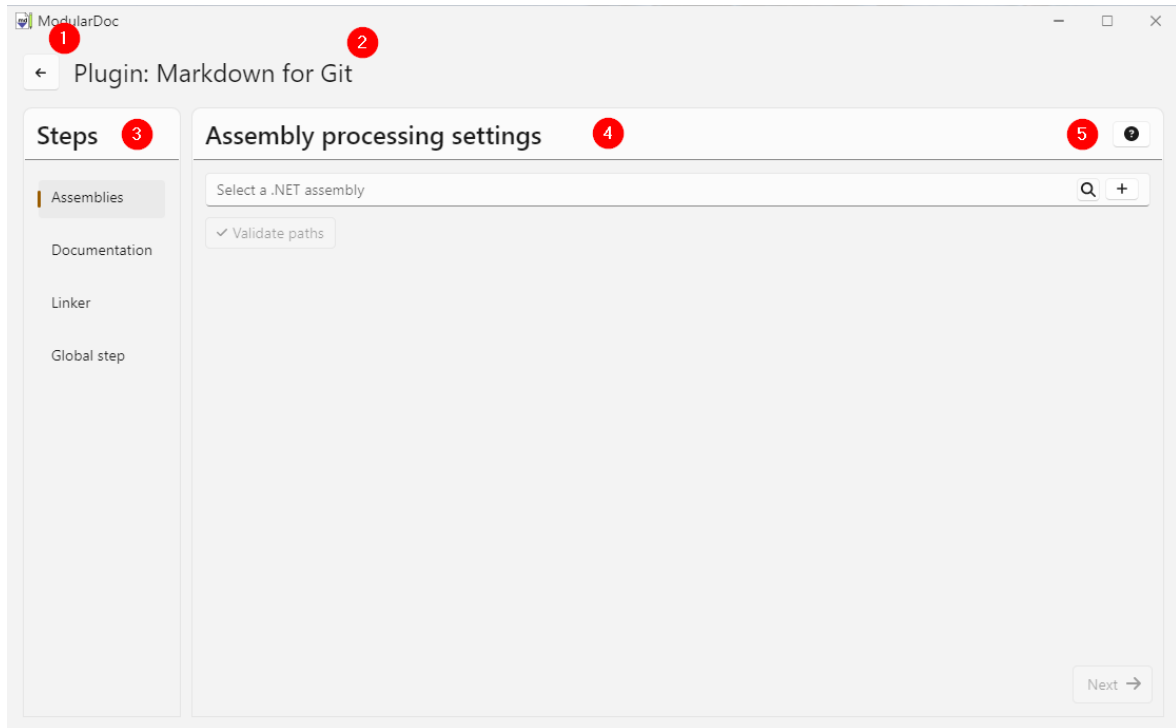


Figure 10.4: ModularDoc configuration wizard

1. Back button for navigating back to the home page
2. Title of the plugin
3. List of steps provided by the plugin, with the current step highlighted
4. The current step configuration title and view
5. Information for the user to help them configure the step correctly

Summary page

Completing the last plugin step navigates to the summary page, which executes the plugin using the created configuration. The execution processes are displayed alongside their progress. Each process logs messages, that can be grouped by the process source and log type. At this stage, it is possible to save the configuration, return to the configuration wizard, or return to the home page.

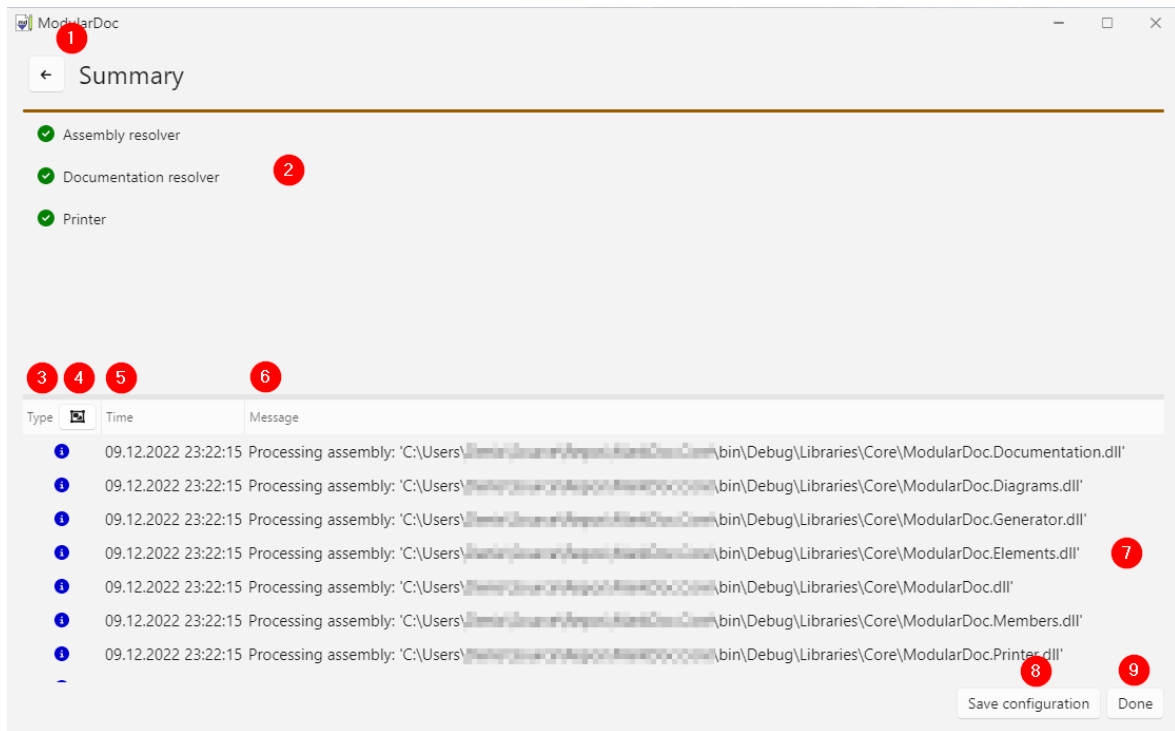


Figure 10.5: ModularDoc summary

1. Back button for navigating back to the configurator
2. List of executed processes, their states
3. Type of logged message
4. Log message grouping options
5. Time of logged message
6. Log message text
7. Log messages
8. Button for saving the created configuration
9. Button for returning back to the home page



Figure 10.6: ModularDoc summary log grouping

1. Toggle for grouping by message source
2. Toggle for grouping by message type
3. Groups by message type
4. Groups by message source

Settings page

The application settings allow the user to toggle its theme between dark and light modes.

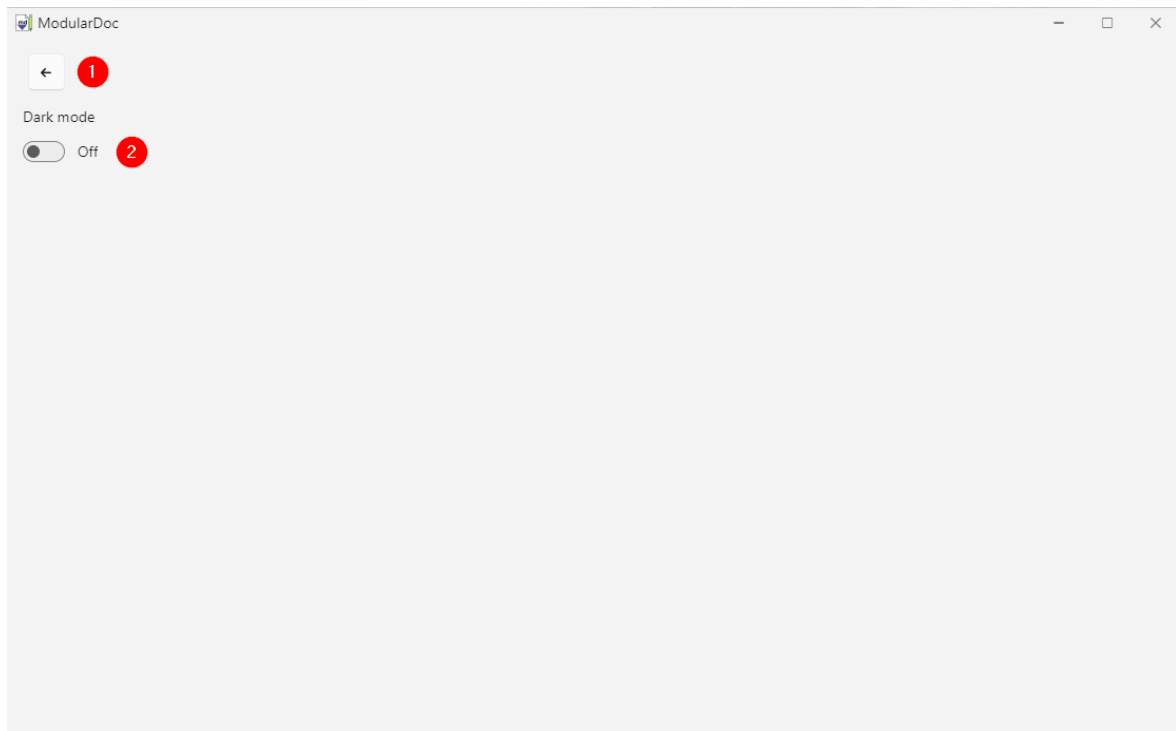


Figure 10.7: ModularDoc settings page

1. Back button for navigating back to the home page
2. Toggle switch for changing the theme from light mode to dark mode

10.1.3 Markdown for Git plugin

The Markdown for Git plugin is designed to process .NET libraries written in C# and output developer documentation in Markdown form suitable for Git platforms such as GitHub or GitLab.

Assemblies step

The assemblies step is designated for selecting file paths to .NET assemblies or executables for processing. It is possible to proceed to the next step only after providing at least one file path.

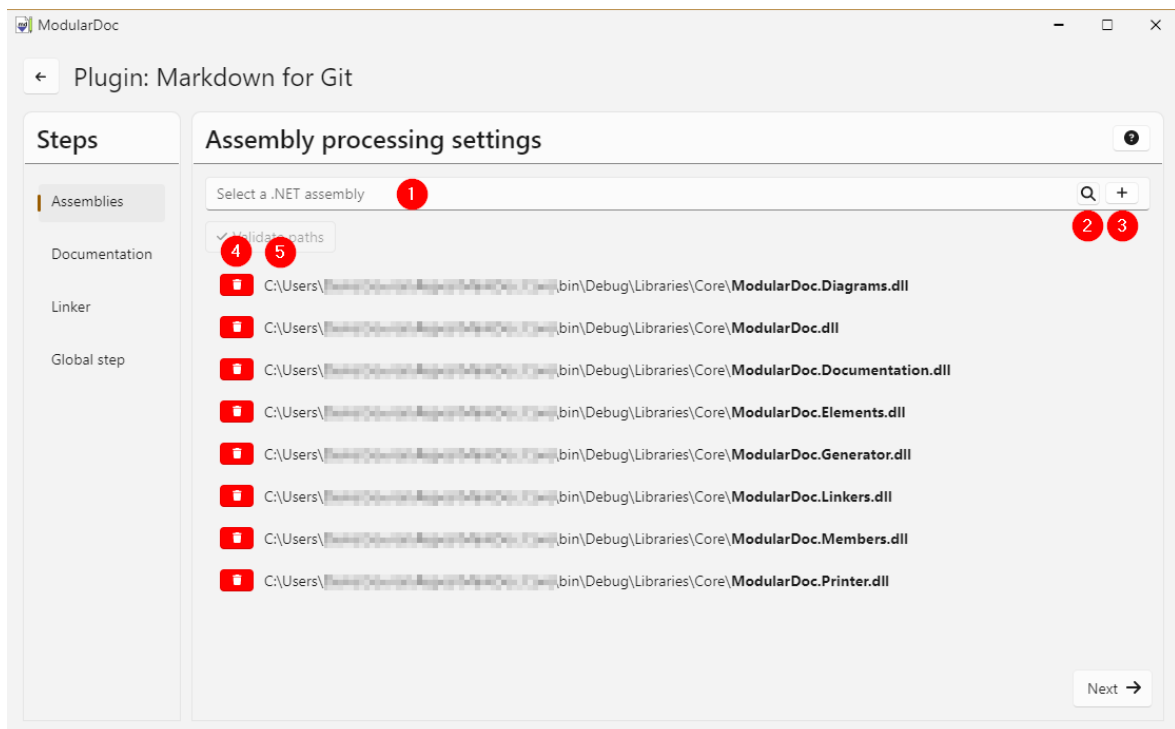


Figure 10.8: Plugin assembly configuration step

1. Input field for supplying a file path
2. Button for opening the file browser and selecting file paths
3. Button for adding the supplied path via the input field, or opening the file browser (see 2) whenever the input field is empty
4. Button for removing the entered file path
5. The path with the filename highlighted

Documentation step

The documentation step checks whether all provided assemblies and executables have their documentation files present. If a documentation file is missing, it is impossible to proceed to the next step. Unfortunately, the version of ModularDoc provided with this thesis does not permit ignoring or finding missing documentation.

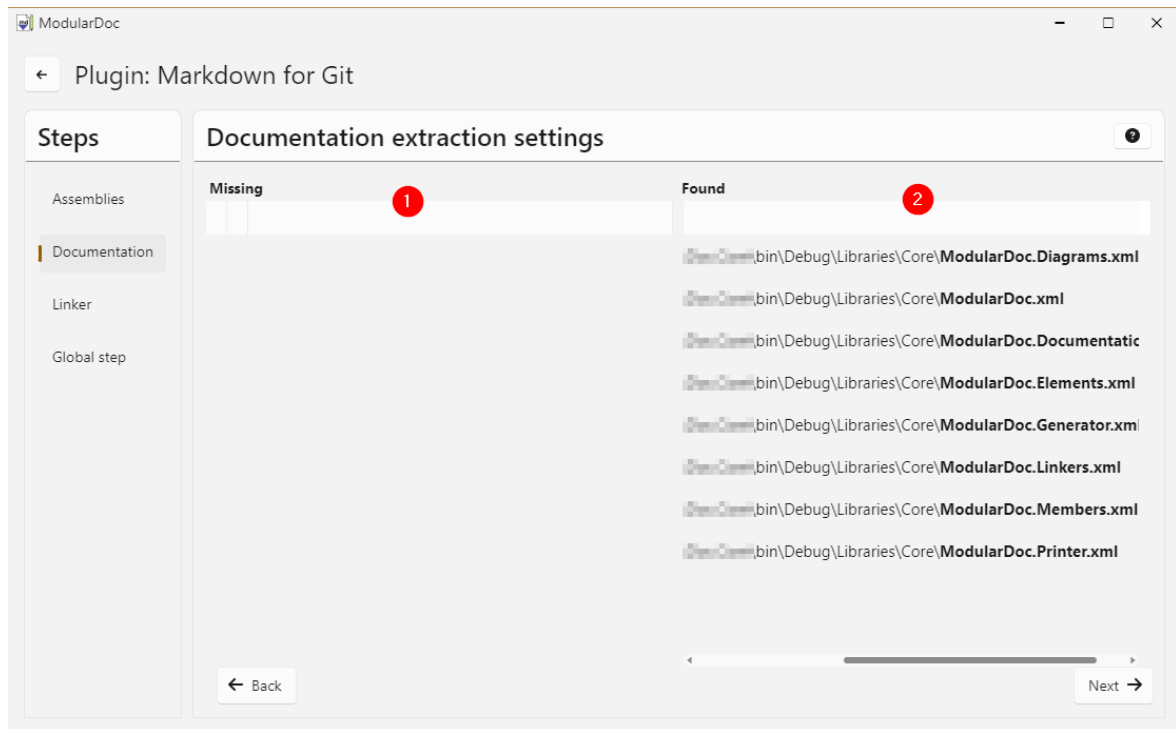


Figure 10.9: Plugin documentation configuration step

1. List of missing documentation files
2. List of automatically discovered documentation files

Linker step

The linker step is designated for configuring linking settings to be valid for the target Git hosting platform.

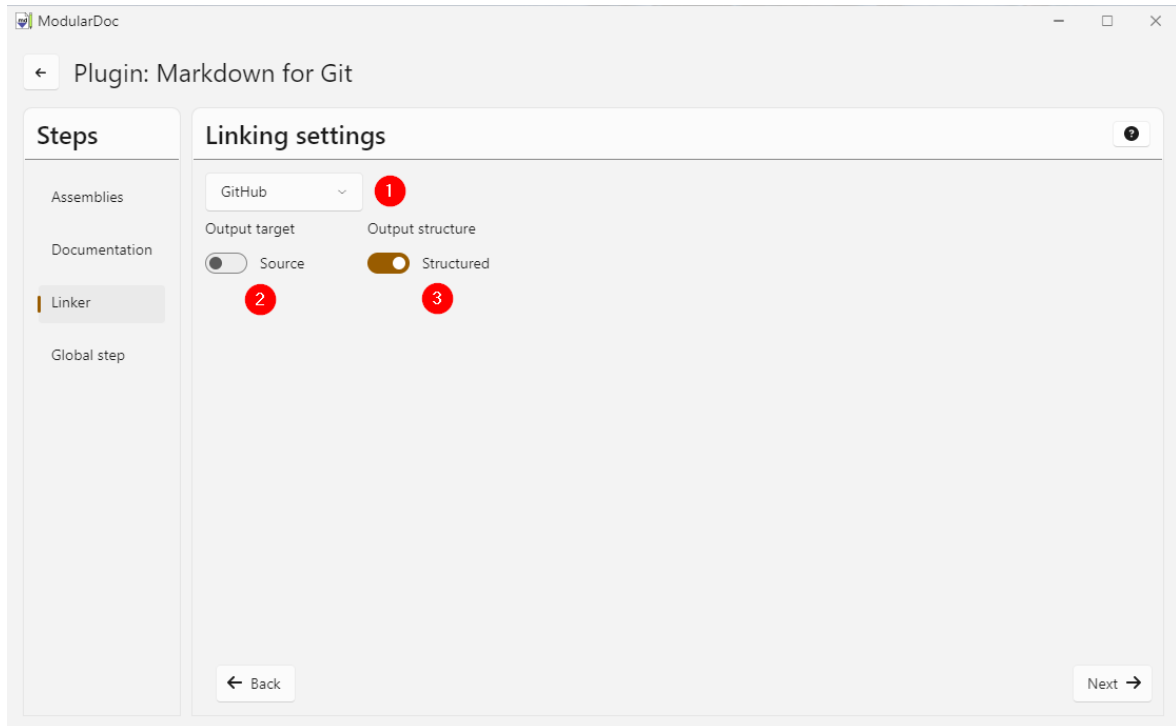


Figure 10.10: Plugin linker configuration step

1. Selection of the target platform - GitHub or GitLab
2. Toggle for the output target - source or wiki
3. Toggle for the output structure - flat or structured

Global step

The global step is designated for configuring both the generated documentation target output paths and ignored namespaces and types. Unchecking a namespace or type ignores it and its children.

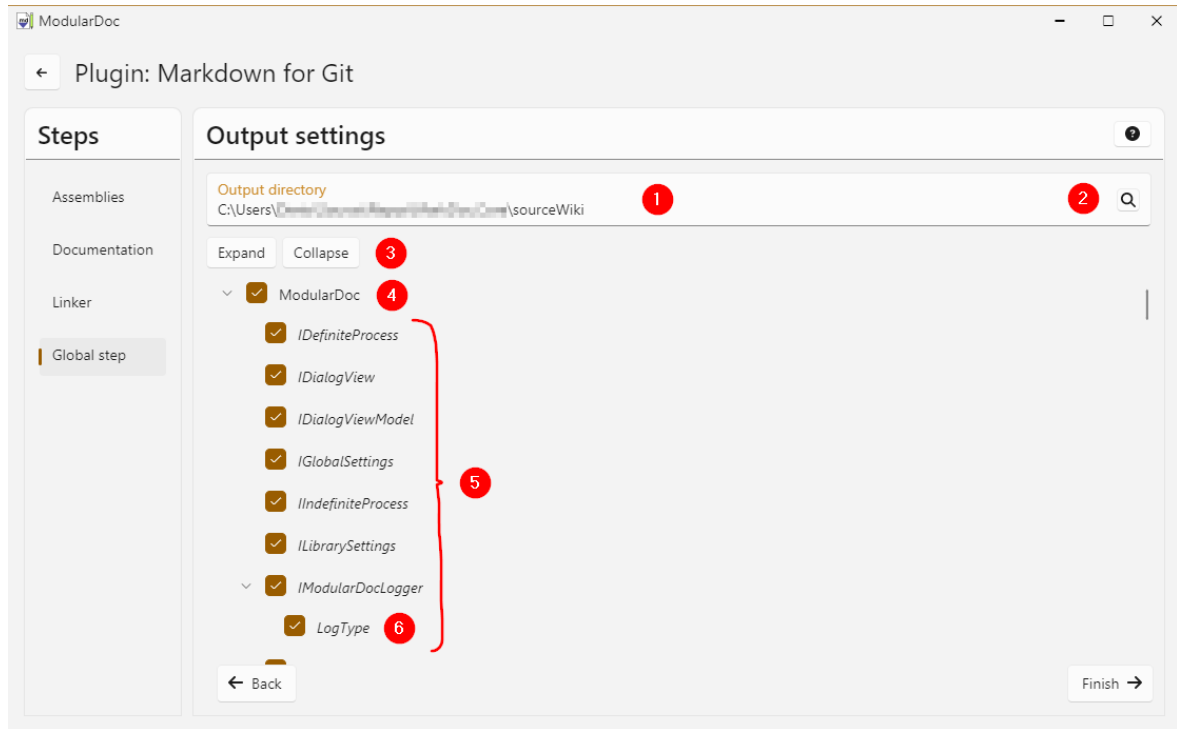


Figure 10.11: Plugin global configuration step

1. Input field for supplying the generated documentation target output path
2. Button for opening the folder browser to select the generated documentation target output path
3. Buttons for either expanding or collapsing the ignore list
4. Namespace entry
5. Type entries
6. Nested type entry

10.1.4 Plugin installation and management

Plugins are installed by placing them in the *Plugins* folder (see figure 10.1). ModularDoc will automatically load plugins upon startup and display them on the home page. ModularDoc can operate without plugins; however, it loses its purpose.

10.2 Developer guide

The developer guide is designated for those who wish to extend ModularDoc with new components or plugins. ModularDoc also allows other areas of the application to be modified; however, due to the complexity and low necessity of these modifications, their description will be omitted from this guide.

10.2.1 Prerequisites

- Visual Studio 2019+, JetBrains Rider 2021.3+, or a text editor such as Visual Studio Code is installed
- .NET SDK 6.0.403+ is installed

10.2.2 Accessing the source code

This thesis is provided with source code for ModularDoc (see ‘Appendix: ModularDoc source code’ on page 87). To access the latest source code, clone it from the project’s repository on GitHub:

Listing 10.1: Script for cloning the ModularDoc project GitHub repository

```
1 git clone https://github.com/hailstorm75/ModularDoc.git
```

Open the *ModularDoc.sln* assembly to start development on the project. Compiling the solution will generate the compiled libraries to the *bin/* folder found in the root of the folder structure (see figure 10.12).

```

ModularDocRepositoryRoot/
├── sourceWiki/
├── src/
│   ├── Components/
│   │   ├── Diagrams/
│   │   ├── Documentation/
│   │   ├── Elements/
│   │   └── ...
│   ├── Libraries/
│   │   ├── Core/
│   │   │   ├── ModularDoc.Composer/
│   │   │   ├── ModularDoc.Constants/
│   │   │   ├── ModularDoc.Core/
│   │   │   ├── ModularDoc.Diagrams/
│   │   │   ├── ModularDoc.Documentation/
│   │   │   ├── ModularDoc.Elements/
│   │   │   ├── ModularDoc.Linker/
│   │   │   ├── ModularDoc.Members/
│   │   │   └── ModularDoc.Printer/
│   │   ├── Helpers/
│   │   ├── ViewModels/
│   │   └── Views/
│   ├── Plugins/
│   ├── ModularDoc.App/
│   ├── ModularDoc.sln
│   └── ...
├── tests/
├── README.md
└── ...

```

Figure 10.12: ModularDoc simplified source code structure

10.2.3 Developing a new component

Follow these steps to develop a new component:

1. Decide on which key area to develop a new component, e.g., element provider
2. Create a new project in the respective folder (in this case, *Components/Elements*) with the following naming scheme - **ModularDoc.Elements.MY_COMPONENT**
3. Configure the project in the same way as one of the other existing provider components
4. Inherit the respective interface from *Libraries/Core* in the new component (in this case, *ModularDoc.Elements*)
5. Based on the interface definitions, implement the new component

10.2.4 Developing a new plugin

Follow these steps to develop a new plugin:

1. Create a new project in the *Libraries/ViewModels* folder with the following naming scheme - **ModularDoc.ViewModels.MY_PLUGIN**
 - (a) Configure the new project in the same way as one of the other existing view model projects
 - (b) Create view models for each defined plugin step and inherit the *BaseStepViewModel<T>* abstract class
 - (c) Create models if necessary
2. Create a new project in the *Libraries/Views* folder with the following naming scheme - **ModularDoc.Views.MY_PLUGIN**
 - (a) Configure the new project in the same way as one of the other existing view projects
 - (b) Create views for each defined plugin step and inherit the *BaseStepUserControl<TVM, TS>* abstract class
3. Create a new project in the *Plugins* folder with the following naming scheme - **ModularDoc.Plugins.MY_PLUGIN**
 - (a) Configure the new project in the same way as one of the other existing plugins
 - (b) Reference the *Libraries/Core/ModularDoc.Core* library
 - (c) Create a type representing the plugin instance, and inherit the plugin interface from the referenced core library, and inherit the *Autofac.Module* class
 - (d) Create types representing plugin configuration steps that inherit the *IPluginStep* interface
 - (e) Register all desired components
 - (f) Register all created view models
 - (g) Register all created views

Summary

The defined goal of this thesis was reached successfully by producing a working custom documentation-generating tool for .NET libraries.

Goal fulfillment

This thesis aimed to create a custom documentation-generating tool for .NET projects using appropriate design patterns (20) and to satisfy user needs for extensibility, ease of use, modern design, support for many output formats, and comparable performance to existing tools.

The resulting project is:

- A set of abstract interfaces defining key parts of the whole application.
- A set of concrete components that are independent of each other.
- A plugin composed of the developed components for generating Markdown documentation.
- A GUI application for hosting plugins and allowing users to configure and execute them.

The decoupled architecture provides maximum extensibility - if a user is missing a feature, they can always add it. The UI/UX of the tool is simple, with hints and labels, making it easy to use. Thanks to the extensibility, the tools unlimited support for output formats; however, the resulting project so far supports Markdown only. Moreover, the tool is very performant and can compete with industry standards such as Doxygen.

Given these facts, the goal of this thesis is fulfilled.

Open-source

The source code for ModularDoc can be found on GitHub, as well as the source for this thesis, as it was written using L^AT_EX:

Project: <https://github.com/hailstorm75/ModularDoc>

Thesis: <https://github.com/hailstorm75/ModularDoc.Thesis>

Personal achievements

Reaching this state of the project is an achievement to be proud of. Considerable effort was put into retaining proper modularity and decoupling business logic from specific frameworks

and platforms. In addition, a broader understanding of documentation and the C# language was gained, alongside experience with solving complex problems without the assistance of documentation or help from fellow developers.

One documented personal goal was to write some of the libraries in F#. The choice to deviate from C# had minor benefits. The produced code is complicated to maintain, specifically in the composer component. The reasons for such a damaging outcome are caused by lack of proper functional programming experience and limited capabilities of core F# features.

ModularDoc source code has flaws, as any software; however, this tool can provide actual value to its users in its current state. And that is an indicator of success.

Future plans

Completing the initial goal was only the beginning. The project needs a suite of tests validating the code's behavior to gain user trust. Furthermore, to gain collaborator interest, the project needs a well-written wiki on how to extend ModularDoc.

By themselves, said tasks are no less complex than creating the tool itself, as both require great care and attention to guarantee positive results. Even if the project does not get the desired awareness from the open-source collaborator community, completing said tasks will still provide valuable experience.

Bibliography

1. AKOPYAN, Denis. *Introduction to .NET reflection - part 1* [NET eXplorer] [online]. 2022-06-18 [visited on 2022-08-13]. Available from: <https://netexplorer.hashnode.dev/exploring-net-reflection-part-1>.
2. ALPHABET, INC. *Google Trends* [Google Trends] [online]. 2022-10-12 [visited on 2022-12-10]. Available from: <https://trends.google.com/trends/explore?q=github,gitlab,bitbucket,sourceforge>.
3. AMAZON WEB SERVICES, INC. *What is DevOps? - Amazon Web Services (AWS)* [Amazon Web Services, Inc.] [online] [visited on 2022-12-11]. Available from: <https://aws.amazon.com/what-is-devops/>.
4. ANDERSON, Rick. *Overview of ASP.NET Core* [Overview of ASP.NET Core] [online]. 2022-03-22 [visited on 2022-03-22]. Available from: <https://docs.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core>.
5. ANDERSON, Tim. *.NET 6 comes to Ubuntu – but where is MAUI for Linux? • DEVCLASS* [DEVCLASS] [online]. 2022-08-18 [visited on 2022-12-10]. Available from: <https://devclass.com/2022/08/18/net-6-comes-to-ubuntu-but-where-is-maui-for-linux/>.
6. ATlassian. *Bitbucket / Git solution for teams using Jira* [Bitbucket] [online] [visited on 2022-12-11]. Available from: <https://bitbucket.org/product>.
7. ATlassian. *Confluence / Your Remote-Friendly Team Workspace* [Atlassian] [online] [visited on 2022-12-11]. Available from: <https://www.atlassian.com/software/confluence>.
8. CARR, Richard. *Adding Lists and Tables to XML Documentation* [Blackwasp] [online]. 2012-04-26 [visited on 2022-02-24]. Available from: <http://blackwasp.co.uk/documentationlists.aspx>.
9. CODE MAZE. *Differences Between .NET Framework, .NET Core, and .NET Standard* [Code Maze] [online]. 2022-01-12 [visited on 2022-12-10]. Available from: <https://code-maze.com/differences-between-net-framework-net-core-and-net-standard/>.
10. CONE, Matt. *Getting Started / Markdown Guide* [Markdown Guide] [online]. In collab. with JARVIS, Dave; KING, Brian. 2022-02-06 [visited on 2022-04-11]. Available from: <https://www.markdownguide.org/getting-started/>.
11. DANIEL, Ch. *Github Users and Growth Statistics: How Many Repos Are There? (2023) / SignHouse* [Usesignhouse] [online]. 2022-11-24 [visited on 2022-12-11]. Available from: <https://www.usesignhouse.com/blog/github-stats>.
12. DEURSEN, Steven Van; SEEMANN, Mark. *Dependency Injection: principles, practices, and patterns* [online]. Revised and expanded edition. Shelter Island, NY: Manning Publications, 2019 [visited on 2022-02-24]. ISBN 978-1-61729-473-0. OCLC: on1082890435.

13. GEORGE, Andy De. *What's new in .NET Core 3.0* [Microsoft Learn] [online]. In collab. with PETRUSHA, Ron; WENZEL, Maira; PASEK, Patryk; SCHONNING, Nick; WARREN, Genevieve; WAGNER, Bill; DYKSTRA, Tom; VICTOR, Youssef; KULIKOV, Petr; COULTER, David; BUCK, Alex; KHAN, Ahson; GOCKE, Andy; AZIZ, Atif; OMELCHUK, Alexander; WRIGHTON, David; METZGAR, Dustin; CANEY, Jeremy; AHO, Jim; DOLLARD, Kathleen; VELURI, Koundinya; ULLRICH, Martin Andreas; MATEO TORRES-RUIZ; WICHER, Krzysztof; SMITH, Steve; SANTOS, Mauricio de los; KARAS, Vitek. 2022-11-03 [visited on 2022-12-10]. Available from: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-core-3-0>.
14. *Get started with syntax analysis (Roslyn APIs)* [Microsoft Learn] [online]. In collab. with WAGNER, Bill; PETRAKIS, George; VICTOR, Youssef; WENZEL, Maira; NOBRE, Lucas; ONDERKA, Petr; GEORGE, Andy De; MRPASCALCASE. 2021-09-15 [visited on 2022-12-10]. Available from: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/get-started/syntax-analysis>.
15. GIT. *Git* [Git] [online]. 2022-03-22 [visited on 2022-03-22]. Available from: <https://git-scm.com/>.
16. GITHUB, INC. *Creating diagrams* [GitHub Docs] [online] [visited on 2022-12-11]. Available from: <https://ghdocs-prod.azurewebsites.net/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams>.
17. GUMSTER, Jason van. *6 places to host your git repository* / *Opensource.com* [Opensource] [online]. 2018-08-30 [visited on 2022-12-10]. Available from: <https://opensource.com/article/18/8/github-alternatives>.
18. HALL, Gary McLean. *Adaptive code: agile coding with design patterns and SOLID principles*. Second edition. Redmond, Washington: Microsoft Press, 2017. Best practices. ISBN 978-1-5093-0258-1. OCLC: ocn964820736.
19. HEESCH, Dimitri van. *Doxygen: Doxygen* [Doxygen] [online]. 2022-10-29 [visited on 2022-12-10]. Available from: <https://www.doxygen.nl/>.
20. HUMBLLOT, Naïma. *Design pattern: what is it and why use it?* [Ryax Technologies] [online]. 2021-08-06 [visited on 2022-12-10]. Available from: <https://ryax.tech/design-pattern-what-is-it-and-why-use-it/>.
21. INTERVIEWBIT. *Bitbucket vs GitHub: Full Comparison* [InterviewBit] [online]. 2022-06-14 [visited on 2022-12-11]. Available from: <https://www.interviewbit.com/blog/bitbucket-vs-github/>.
22. JIRICEK, Zenobius. *Why is Bitbucket from Atlassian not as popular as GitHub for hosting code repositories and version control?* [Quora] [online]. 2022-04-03 [visited on 2022-12-10]. Available from: <https://www.quora.com/Why-is-Bitbucket-from-Atlassian-not-as-popular-as-GitHub-for-hosting-code-repositories-and-version-control>.
23. KATZ, Max. *MVVM Architecture* [online]. In collab. with SYCHEVSKYI, Ihor; WALMSLEY, Dan; KIRK, Steven; VESKE, Engincan. 2022-04-06 [visited on 2022-12-10]. Available from: <https://docs.avaloniaui.net/guides/basics/mvvm>.

24. LABVIEW. *Factory pattern - LabVIEW Wiki* [LabVIEW Wiki] [online]. 2020-09-06 [visited on 2022-12-11]. Available from: https://labviewwiki.org/wiki/Factory_pattern.
25. LANDER, Rich. *Introducing .NET 5* [.NET Blog] [online]. 2019-06-05 [visited on 2022-12-10]. Available from: <https://devblogs.microsoft.com/dotnet/introducing-net-5/>.
26. LANDWERTH, Immo. *The future of .NET Standard* [.NET Blog] [online]. 2020-09-15 [visited on 2022-12-10]. Available from: <https://devblogs.microsoft.com/dotnet/the-future-of-net-standard/>.
27. *Mermaid documentation* [mermaid] [online]. In collab. with VINOD, Sidharth; SPIER, Sebastian; BOTTERO, Emerson; SVEIDQVIST, Knut; KLINK, Alois. 2022-02-24 [visited on 2022-02-24]. Available from: <https://mermaid-js.github.io/mermaid/#/>.
28. MICROSOFT. *Microsoft .NET Framework - Microsoft Lifecycle* [Microsoft Learn] [online]. 2022-10-12 [visited on 2022-12-10]. Available from: <https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-framework>.
29. MICROSOFT. *Azure DevOps Services / Microsoft Azure* [online] [visited on 2022-12-11]. Available from: <https://azure.microsoft.com/en-us/products/devops/>.
30. MICROSOFT. *Microsoft Design* [Microsoft Design] [online] [visited on 2022-12-11]. Available from: <https://www.microsoft.com/design/fluent/>.
31. MICROSOFT. *NuGet Gallery / Home* [NuGet Gallery | Home] [online]. 2022-03-22 [visited on 2022-03-22]. Available from: <https://www.nuget.org/>.
32. MICROSOFT. *What is .NET? An open-source developer platform.* [What is .NET? An open-source developer platform.] [online]. 2022-03-22 [visited on 2022-03-22]. Available from: <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet>.
33. OSENKOV, Kirill. *SourceBrowser* [GitHub] [online]. 2022-04-12 [visited on 2022-12-10]. Available from: <https://github.com/KirillOsenkov/SourceBrowser/blob/2b57a3d551d02c05b90d006f48cfa07c0a7e5cd2/README.md>. original-date: 2015-09-24T04:55:42Z.
34. PETRAKIS, George. *Vsxdm/README.md at master · lijunle/Vsxdm* [GitHub] [online]. 2022-10-11 [visited on 2022-12-10]. Available from: <https://github.com/lijunle/Vsxdm/blob/master/README.md>.
35. RACHEL, Tomer Ben. *Why documentation matters, and why you should include it in your code* [freeCodeCamp.org] [online]. 2018-07-08 [visited on 2022-12-10]. Available from: <https://www.freecodecamp.org/news/why-documentation-matters-and-why-you-should-include-it-in-your-code-41ef62dd5c2f/>.
36. RANDAL, Colin. *ASCII Art frequently asked questions* [ASCII Art Archive] [online]. 2015-06-17 [visited on 2022-12-10]. Available from: <https://www.asciart.eu>.

37. RUPARELIA, Sital. *Stop Over-Planning and Over-Thinking Your Career / LinkedIn* [Stop Over-Planning and Over-Thinking Your Career] [online]. 2016-01-27 [visited on 2022-04-16]. Available from: <https://www.linkedin.com/pulse/stop-over-planning-over-thinking-your-career-sital-ruparelia/>.
38. SMRITA. *Benefits of extracting documentation from software code* [Sigasi] [online]. 2014-10-17 [visited on 2022-12-10]. Available from: <https://insights.sigasi.com/opinion/benefits-extracting-documentation-software-code/>.
39. STILGAR. *The State of .NET GUI Frameworks* [Sietch] [online]. 2018-08-20 [visited on 2022-12-10]. Available from: <https://sietch.net/ViewNewsItem.aspx?NewsItemID=271>.
40. *TIOBE Index* [TIOBE] [online]. 2022-10-12 [visited on 2022-12-10]. Available from: <https://www.tiobe.com/tiobe-index/>.
41. VARUS, Jaanus. *discosultan/vsdot-2-md* [GitHub] [online]. 2022-10-16 [visited on 2022-12-10]. Available from: <https://github.com/discosultan/vsdot-2-md/blob/788631a52b2bd0e626e0c3b24fe5efba4134a915/README.md>. original-date: 2016-05-10T09:30:13Z.
42. WEI, Liangying. *DocFX - static documentation generator / DocFX website* [GitHub] [online]. In collab. with DEBUTTS, Sean; WENZEL, Maira; JIN, Yisheng; DELIMARSKY, Den; YU, Renze. 2020-09-22 [visited on 2022-12-10]. Available from: <https://dotnet.github.io/docfx/>.
43. *What is Windows Forms - Windows Forms .NET* [Microsoft Learn] [online]. In collab. with GEORGE, Andy (Steve) De; BUCK, Alex. 2022-09-22 [visited on 2022-12-10]. Available from: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/>.
44. WOODWARD, Martin. *Include diagrams in your Markdown files with Mermaid* [The GitHub Blog] [online]. 2022-02-14 [visited on 2022-12-11]. Available from: <https://github.blog/2022-02-14-include-diagrams-markdown-files-mermaid/>.
45. *XML documentation comments - document APIs using /// comments* [Microsoft Docs] [online]. In collab. with WAGNER, Bill; WARREN, Genevieve. 2022-12-01 [visited on 2022-04-05]. Available from: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/xml/doc/>.

Appendices

Appendices include files created in the context of this thesis.

Appendix: ModularDoc compiled

Compiled source code (see ‘Appendix: ModularDoc source code’ on page 87) of the resulting application of this thesis - ModularDoc.

Prerequisites and instructions for running the compiled application are provided in ‘User guide’ on page 67.

Appendix: ModularDoc source code

Source code of the resulting application of this thesis - ModularDoc. It is written in C# and F# on the .NET 6 platform.

The provided source is a snapshot of its version-controlled state hosted on GitHub. The state was captured on the 11th of December, 2022, and corresponds to the following Git commit identifier: *f6af81c59480962fa1fb3c39f3c47e8be4f634be*.

Instructions for modifying the source code, alongside instructions on how to get its latest version-controlled state, are provided in ‘Accessing the source code’ on page 78.